"No 'deployed' claim without digest identity, smoke evidence, and a rehearsed rollback."

MODULE 51

Capstone Security, CI/CD and Deployment

Ship a secured, gated release of the Northstar CRM: JWT deny-by-default authorization, security scanning, an immutable Docker image, a GitHub Actions pipeline, and a k3s deployment with smoke tests and a rehearsed rollback.


TEKsystems
Global Services
WORKFORCE
DEVELOPMENT


TEKsystems
Global Services
WORKFORCE
DEVELOPMENT

MODULE 51 OVERVIEW

Ship a secured, digest-pinned CRM release through gated CI/CD to k3s, with authenticated smoke, deny-path proof, and a rehearsed rollback.

Feature-complete is not release-complete. This module hardens JWT/RBAC, gates the pipeline with security scans, and proves deploy, smoke, and rollback against the architecture and backend built in Modules 48-50.

DURATION & LAB

4 Hours Theory

JWT/RBAC hardening, CORS/CSRF/actuator protection, SAST/DAST and dependency scanning, container and k3s deployment review.

2 Hours Lab

lab51-crm: deny-by-default authorization, gated pipeline, digest-pinned image, k3s deploy with probes, authenticated and 401/403 smoke, rollback.

Scenario

Ship a release candidate for Amina (CUS-1001) with authenticated smoke passing and anonymous calls returning 401.

MODULE ARC

Deny By Default

JWT resource server first -- every endpoint requires a role unless explicitly permitted.

Immutable Images

Digest-pinned, non-root, multi-stage builds -- never deploy :latest.

Rollback Rehearsed

A previous-digest rollback is proven before it's ever needed in production.

KEY TAKEAWAY Total time: 4 hours theory + 2 hours lab = 6 hours. No 'deployed' claim without digest identity, authenticated and deny-path smoke evidence, and a rehearsed rollback.

WHY DEPLOYMENT READINESS MATTERS

A working feature is not a shippable release -- access control, scanning, image provenance, and recovery must be evidenced together.

Feature-Complete Is Not Release-Complete

Leadership freezes merge/promote until security, delivery, provenance, and recovery are proven.

Open Endpoints Fail the Course

An open /api/** in a shared cluster is a named, non-negotiable security failure.

Immutable or It Didn't Ship

Floating :latest with no digest means no one can prove what is actually running.

Graded Like Every Lab

Lab 51 is scored on the same 100-point rubric: core impl, integration, failure handling, verification, security, docs.

KEY TAKEAWAY Floating :latest without a digest loses integration marks; skipping the unauthorized smoke test is a named failure-handling deduction.

WHY DEPLOYMENT READINESS MATTERS

Deployment readiness — Ensures that software is thoroughly tested, secure, stable, and operationally prepared for a successful release to production.

KEY REASONS

1. **REDUCES RISKS**
   Minimizes the chances of failures, outages, and data loss.
2. **ENSURES QUALITY**
   Validates functionality, performance, and security standards.
3. **IMPROVES RELIABILITY**
   Ensures the system performs as expected in real-world conditions.
4. **ENHANCES TRUST**
   Builds confidence among users, business stakeholders, and teams.
5. **SUPPORTS BUSINESS GOALS**
   Enables faster releases and supports business agility and growth.
6. **REDUCES COSTS**
   Prevents costly incidents, rollbacks, and hotfixes.

WHAT IT ENABLES

CONFIDENT RELEASES → SMOOTHER DEPLOYMENTS → BETTER SYSTEM STABILITY → FEWER INCIDENTS & DOWNTIME → IMPROVED USER EXPERIENCE → STRONGER COMPETITIVE ADVANTAGE

Being deployment ready transforms potential risk into reliable delivery and business value.

MODULE LEARNING OBJECTIVES

By the end of this module, you will be able to ship a secured, gated release of the Northstar CRM.

1. Apply JWT authorization with deny-by-default request matching.
2. Protect secrets, headers, and actuator exposure.
3. Build gated CI/CD stages with verified artifact identity.
4. Publish immutable, multi-stage, non-root container images.
5. Deploy safely to Kubernetes (k3s) with health probes.
6. Execute authenticated and unauthorized smoke tests.
7. Prove rollback to a previous image digest.
8. Document release evidence for the Module 52 defense.

KEY TAKEAWAY These eight objectives are drawn directly from the graded lab -- you will build and prove every layer of the Northstar CRM's release gate.

SECURITY TESTING REVIEW

A quick review of the security-testing vocabulary this module puts into practice against the real Northstar CRM.

Authorization Tests

MockMvc with a JWT stub proves 401/403/200 for anonymous, wrong-role, and correct-role calls.

Static vs. Dynamic

SAST reads source without running it; DAST probes a running instance from the outside.

Vulnerability Assessment

Dependency, secret, and image scans each catch a different class of exposure.

Remediation, Not Just Detection

A finding without a fix or a time-bound exception is not actually handled.

KEY TAKEAWAY Security testing is a set of layered checks, not one tool -- authorization tests, SAST, DAST, and scanners each catch what the others miss.

SECURITY TESTING REVIEW

OBJECTIVE — Identify vulnerabilities, validate security controls, and reduce risk before release.

Protect Assets | Prevent Threats | Ensure Compliance | Build Trust

SECURITY TESTING APPROACH

1. PLAN
- Define scope and objectives
- Identify assets and risks
- Select testing types
- Define success criteria

2. SCAN
- Static Application Security Testing (SAST)
- Dependency & License Scan
- Infrastructure Scan
- Secrets & Configuration Scan

3. ASSESS
- Analyze vulnerabilities
- Validate and prioritize
- Assess business impact
- Map to risk rating (CVSS)

4. TEST
- Dynamic Application Security Testing (DAST)
- API Security Testing
- Authentication Testing
- Authorization Testing
- Penetration Testing

5. REMEDIATE
- Fix vulnerabilities
- Re-test and verify
- Track and close issues
- Update security controls

6. REPORT
- Summary of findings
- Risk and impact
- Remediation status
- Recommendations

SECURITY TEST COVERAGE

APPLICATION
- Input Validation
- Session Management
- Error Handling
- Business Logic

API
- Authentication
- Authorization
- Rate Limiting
- Data Exposure

AUTHENTICATION
- Weak Passwords
- MFA Verification
- Account Lockout
- Token / JWT Security

AUTHORIZATION
- Access Control
- Privilege Escalation
- IDOR / BOLA
- Sensitive Data Access

INFRASTRUCTURE
- Network Security
- Configurations
- Encryption
- Secrets Management

COMPLIANCE
- OWASP Top 10
- GDPR / PII
- PCI-DSS
- Industry Standards

TESTING TYPES

SAST — Find issues in source code early.

DAST — Test running application from outside.

IAST — Analyze app behavior in runtime.

PEN TEST — Simulate real attacks to find exploitable flaws.

RISK SEVERITY

CRITICAL — Immediate risk to system

HIGH — High impact, needs fix

MEDIUM — Moderate impact, plan to fix

LOW — Low impact, monitor

Security testing is continuous. Test early, test often, and ship secure.

TRY IT YOURSELF -- EXERCISE 2: PLAN RBAC NEGATIVE TESTS

Reinforces: planning anonymous, wrong-role, and correct-role test cases before writing a single Spring Security rule.

STEPS (notes/lab51-rbac-negative-plan.md)

Step	What To Do
1 -- Cases	Anonymous GET, AGENT calling a MANAGER-only action, AGENT correct read.
2 -- Check the Reference	Anonymous -> 401; wrong role -> 403; correct role -> 200 -- three distinct classes.
3 -- Endpoint Map	List every /api/** endpoint and its minimum required role.
4 -- Scope	Test plan only -- SecurityConfig implementation happens during Lab 51.

RBAC negative-test skeleton

```
anonymousCustomersUnauthorized() -> 401.
deleteCustomerRequiresManagerRole() (AGENT) -> 403.   agentCanReadCustomers() -> 200.
```

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 2 before continuing: exercises/exercise-02-rbac-negative-plan.md (workspace: examples/module-51-exercises).

SAST -- STATIC APPLICATION SECURITY TESTING

Read the source without running it -- SAST catches patterns a human reviewer can miss at scale.

WHAT SAST CATCHES

Hardcoded secrets or credentials in source.

Injection-prone patterns (SQL, command, path).

Insecure defaults left in configuration files.

Dependency-declared code with known unsafe patterns.

RUNNING SAST HERE

- Runs as a required CI/CD gate, not an optional local check.
- Findings get triaged -- fixed or given a time-bound exception.
- A SAST failure must be able to fail the pipeline, not just warn.
- Instructor-named tooling: OWASP Dependency-Check, Trivy, gitleaks, Snyk, or similar.

Pipeline gate order (excerpt)

```
build -> verify -> secret-scan -> dep-scan -> image-build -> image-scan -> publish -> deploy
# do not deploy unscanned or unverified artifacts
```

KEY TAKEAWAY A SAST gate that only warns, and never actually fails the build, is not a gate -- it is a suggestion nobody is required to read.

SAST Review

Static Application Security Testing (SAST) analyzes source code, bytecode, or binaries without executing the application to find security vulnerabilities early.

SAST PROCESS

1. **SOURCE CODE INPUT** — Developers commit code to repository.
2. **CODE SCANNING** — SAST tool analyzes code statically without execution.
3. **VULNERABILITY DETECTION** — Identifies security flaws, bad practices, and misconfigurations.
4. **RESULTS & REPORTING** — Generates detailed reports with findings and recommendations.
5. **REMEDIATION** — Developers fix issues and improve code quality.
6. **RE-SCAN & VERIFY** — Re-scan to verify fixes and ensure secure code.

WHAT SAST DETECTS

- **Injection Flaws**
  SQL, Command, LDAP, XPath
- **Broken Authentication**
  Hardcoded credentials, weak passwords
- **Sensitive Data Exposure**
  Hardcoded secrets, PII leakage
- **Security Misconfigurations**
  Insecure headers, debug code, open permissions
- **Code Quality & Best Practices**
  Dead code, null dereference, resource leaks

SAST ANALYSIS CAPABILITIES

- Data flow analysis
- Control flow analysis
- Taint analysis
- Pattern matching
- OWASP Top 10 coverage
- Custom rules & policies
- Multi-language support
- IDE & CI/CD integration

INTEGRATION POINTS

- **Source Code Repositories**
  GitHub, GitLab, Bitbucket, Azure DevOps
- **CI/CD Pipelines**
  Jenkins, GitHub Actions, GitLab CI, Azure Pipelines
- **IDE Plugins**
  VS Code, IntelliJ IDEA, Eclipse
- **Issue Tracking**
  Jira, Azure Boards, Bugzilla
- **Reporting & Dashboards**
  Centralized visibility and metrics

BEST PRACTICES

- Integrate SAST early in the SDLC
- Define and enforce quality gates
- Use custom rules for your codebase
- Train developers on secure coding
- Track metrics and trends
- Continuously improve and re-assess

DAST -- DYNAMIC APPLICATION SECURITY TESTING

Probe a running instance from the outside -- DAST finds what only shows up once the application is actually live.

WHAT DAST CATCHES

Real 401/403/200 behavior against live, running endpoints.

Response headers that leak information (server version, stack traces).

Actuator endpoints that are accidentally public.

CORS misconfiguration only visible from an actual browser-origin request.

DAST VS. THE MANUAL PROBES

- Overlaps with the smoke tests covered later this module, on purpose.
- Catches runtime configuration drift SAST cannot see in source.
- Runs against a deployed instance -- staging or the training cluster.
- Findings still require a fix or a documented, time-bound exception.

Smoke-style DAST probe (excerpt)

```
curl -fsS "$CRM_URL/actuator/health/readiness"
test "$(curl -s -o /dev/null -w '%{http_code}' "$CRM_URL/api/customers")" = "401"
```

KEY TAKEAWAY A SAST-clean codebase can still fail DAST if the deployed configuration drifts from what the source actually intended.

DAST REVIEW

Dynamic Application Security Testing (DAST) tests a running application from the outside to find vulnerabilities in real time.

DAST PROCESS

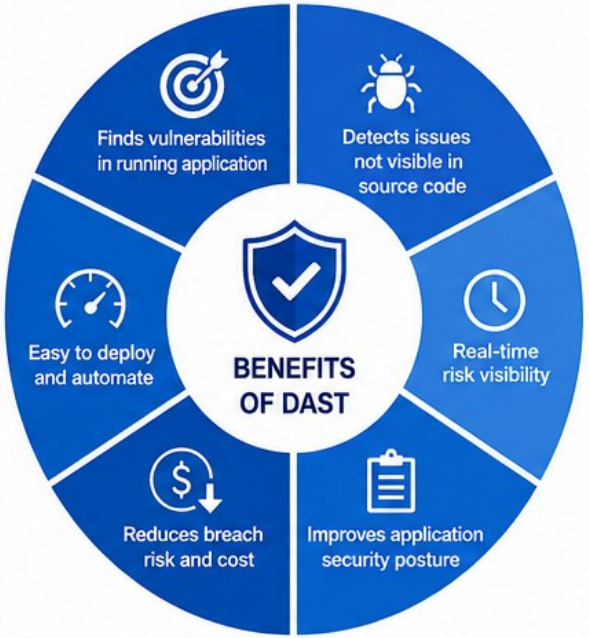
1. **PLAN & SCOPE** — Define objectives, scope, target URLs, and test environment.
2. **DISCOVERY** — DAST tool crawls the application to discover endpoints and links.
3. **VULNERABILITY DETECTION** — Send malicious inputs and analyze responses to detect vulnerabilities.
4. **ANALYSIS & CORRELATION** — Correlate findings, remove false positives, and assess risk severity.
5. **REPORTING** — Generate detailed reports with evidence, impact, and remediation.
6. **REMEDIATION & RETEST** — Developers fix issues. Re-test to verify and ensure they are resolved.

WHAT DAST DETECTS

- **Injection Flaws**
  SQL Injection, NoSQL Injection, Command Injection
- **Authentication Issues**
  Broken Authentication, Weak Session Management
- **Cross-Site Scripting (XSS)**
  Reflected, Stored, DOM-based
- **Insecure Direct Object References (IDOR)**
  Unauthorized access to objects
- **Security Misconfigurations**
  Exposed admin pages, verbose errors, missing security headers
- **Sensitive Data Exposure**
  Unencrypted data in transit, directories, backup files

DAST TESTING CAPABILITIES

- Crawling & endpoint discovery
- Automated attack simulation
- Business logic testing
- Authentication & session testing
- AJAX & web service testing
- HTML5 & JavaScript testing
- API testing (REST/SOAP/GraphQL)
- OWASP Top 10 coverage
- Custom attack & test policies
- Continuous scanning support
- CI/CD integration



INTEGRATION POINTS

- **Web Applications**
  Internal, External, Staging, Production
- **CI/CD Pipelines**
  Jenkins, GitHub Actions, GitLab CI, Azure Pipelines
- **DAST Tools**
  OWASP ZAP, Burp Suite, Acunetix, Qualys, Veracode
- **Source Code Repositories**
  GitHub, GitLab, Bitbucket, Azure DevOps
- **Reporting & Dashboards**
  Centralized visibility and metrics
- **Issue Tracking**
  Jira, Azure Boards, Bugzilla

BEST PRACTICES

- Define clear scope and objectives
- Run DAST early and regularly
- Tune scans to reduce false positives
- Combine with SAST and manual testing
- Track metrics and trends
- Re-test after remediation

VULNERABILITY ASSESSMENT

Dependency, secret, and image scans each surface a different exposure -- run all three, not just one.

THREE SCAN TYPES

Dependency scan: known-vulnerable libraries by CVE.

Secret scan: credentials or tokens committed to Git history.

Image scan: OS packages and layers inside the built container.

Each catches a class of risk the other two cannot see.

ASSESSMENT DISCIPLINE

- Critical findings block the pipeline unless explicitly exceptioned.
- Exceptions are time-bounded, with an owner and expected fix date.
- Ignoring a critical CVE silently is a named failure -- fix or document it.
- Scan reports are sanitized before they are linked in evidence docs.

Digest inspection (from the lab guide)

```
docker image inspect "$REGISTRY/crm-api:${VERSION}-${GIT_SHA}" \
  --format='{{index .RepoDigests 0}}'
```

KEY TAKEAWAY Ignoring a critical CVE without creating an honest, owned risk entry is explicitly named as a failure this module does not accept.

VULNERABILITY ASSESSMENT

A systematic process to identify, evaluate, and prioritize vulnerabilities in systems, applications, and networks.

ASSESSMENT PROCESS

1. **DEFINE SCOPE** — Identify assets, systems, and boundaries.
2. **INFORMATION GATHERING** — Collect data about the environment and technologies.
3. **VULNERABILITY IDENTIFICATION** — Scan and discover vulnerabilities using tools and techniques.
4. **VULNERABILITY ANALYSIS** — Analyze findings, validate, and understand potential impact.
5. **RISK EVALUATION** — Assess risk based on likelihood and impact.
6. **PRIORITIZATION** — Rank vulnerabilities based on risk and business context.
7. **REPORTING & REMEDIATION** — Report findings and recommend fixes. Track remediation.

WHAT IT DETECTS

- **Missing Patches** — Outdated software and unapplied security updates
- **Misconfigurations** — Weak or incorrect system and security settings
- **Weaknesses** — Design flaws, insecure protocols, and services
- **Access Issues** — Weak passwords, excessive permissions, and exposed accounts
- **Network Issues** — Open ports, vulnerable services, and network exposures

ASSESSMENT COMPONENTS

- **Assets** — Hardware, software, applications, databases, cloud resources
- **Threats** — Potential threats that could exploit vulnerabilities
- **Vulnerabilities** — Known weaknesses that can be exploited
- **Impact** — Business, operational, financial, and compliance impact
- **Likelihood** — Probability of a vulnerability being exploited

RISK RATING (CVSS EXAMPLE)

9.0 - 10.0	CRITICAL	Immediate attention required
7.0 - 8.9	HIGH	Fix as soon as possible
4.0 - 6.9	MEDIUM	Fix in normal maintenance
0.1 - 3.9	LOW	Monitor and fix when possible
0.0	INFO	Informational findings

BEST PRACTICES

- Assess regularly and continuously
- Keep systems and tools updated
- Involve cross-functional teams
- Validate and verify all findings
- Track remediation and re-assess
- Prioritize based on risk and impact
- Maintain accurate asset inventory

SECURITY REMEDIATION WORKFLOW

A finding is not resolved until it is fixed or exceptioned -- and both outcomes get recorded the same way.

REMEDIATION PATHS

Fix: upgrade the dependency, rotate the secret, patch the config.

Exception: owner, business justification, and an expiry date.

Re-scan after any fix to confirm the finding is actually gone.

Never disable a scanner to force a pipeline green.

DOCUMENTING THE OUTCOME

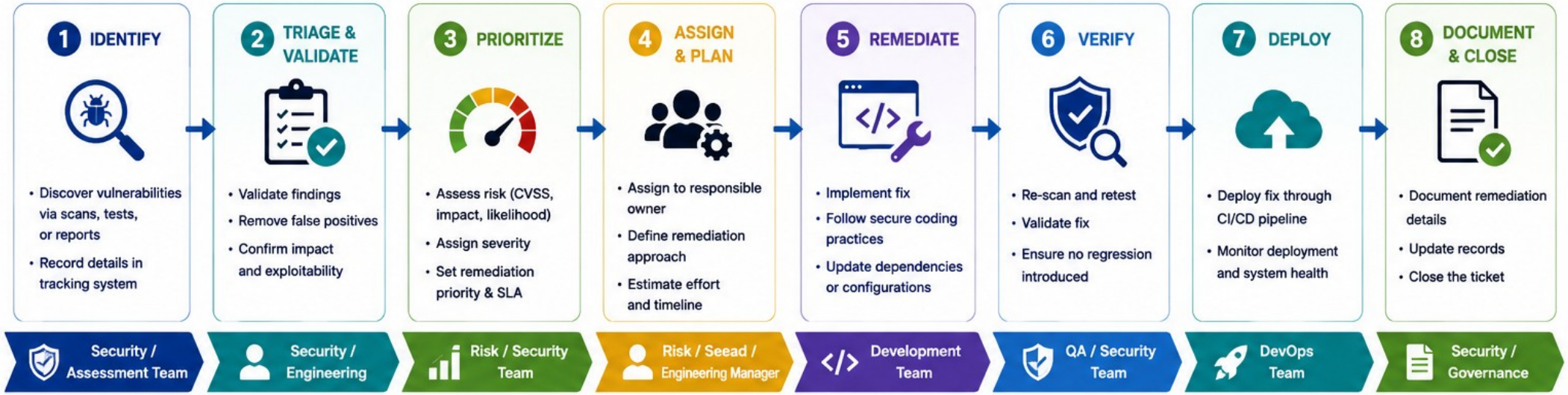
- Every finding's disposition lives in reports/ or docs/, sanitized.
- A residual-risk entry names who owns the follow-up.
- Remediation notes feed directly into Module 52's evidence index.
- Silence on a known finding is treated the same as denying it existed.

Exception record (concept)

```
## Exception: CVE-2024-XXXXX
Owner: platform-team   Reason: no fixed version available yet
Expiry: 2026-09-01   Compensating control: WAF rule + monitoring
```

KEY TAKEAWAY Disabling a scanner to force a green pipeline is explicitly the kind of shortcut this module's grading is designed to catch.

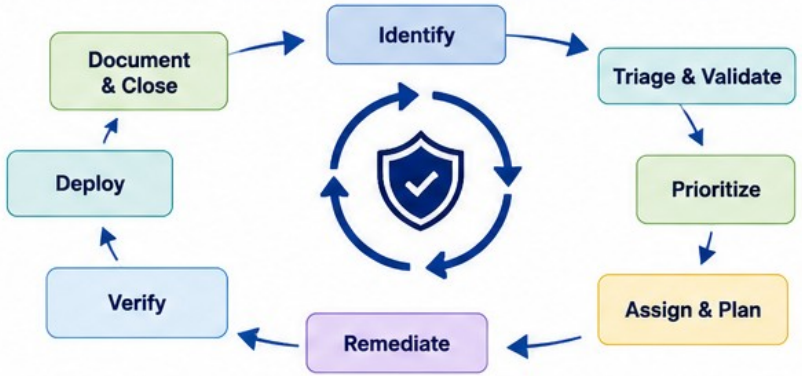
SECURITY REMEDIATION WORKFLOW



INPUT SOURCES

- Vulnerability Scanners SAST, DAST, IA, VA
- Penetration Testing
- Dependency Scanners
- Code Reviews
- Bug Bounty / Reports
- Logs & Monitoring

WORKFLOW OVERVIEW



OUTPUT & BENEFITS

- Reduced risk and attack surface
- Improved security posture
- Compliance and audit readiness
- Faster remediation cycle
- Better visibility and accountability

KEY PRINCIPLES

Risk-Based Prioritization | Timely Remediation | Clear Ownership & Accountability | Automation Where Possible | Continuous Verification | Documentation & Traceability | Continuous Improvement

TRY IT YOURSELF -- EXERCISE 1: CAPSTONE THREAT CHECKLIST

Reinforces: naming the CRM's top threats concretely, using synthetic fixtures, not a generic OWASP paste.

STEPS (notes/lab51-threat-checklist.md)

Step	What To Do
1 -- Threats	Broken authz on customer IDs, secret leakage, vulnerable deps, mutable tags, failed rollback.
2 -- Check the Reference	Lab 51 combines JWT/RBAC, pipeline SAST, immutable images, k3s, smoke/rollback.
3 -- Fixtures	Negative tests use synthetic IDs (CUS-1001) -- never a real customer.
4 -- Scope	Checklist only -- SecurityConfig and pipeline work happen during Lab 51.

Threat checklist skeleton

```
Threats: broken authz, secret leakage, vulnerable deps, mutable :latest, failed rollback.
Each threat maps to a control + a test, per docs/threat-model.md.
```

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 1 before continuing: exercises/exercise-01-threat-checklist.md (workspace: examples/module-51-exercises).

DOCKER ARCHITECTURE REVIEW

A quick review of the container shape this module ships -- multi-stage, non-root, and digest-identified.

Multi-Stage Build

A build stage compiles; a slim runtime stage ships -- no build tooling in production.

Non-Root User

The container runs as an unprivileged USER, never root, inside the shared cluster.

Immutable Tag

crm-api:<version>-<gitsha> -- never a floating :latest as the only identity.

Digest Recorded

The exact pushed image digest gets written into docs/security-deploy-demo.md.

KEY TAKEAWAY Rebuilding a slightly different image per environment destroys provenance -- build once, push once, deploy that exact digest everywhere.

DOCKER ARCHITECTURE REVIEW

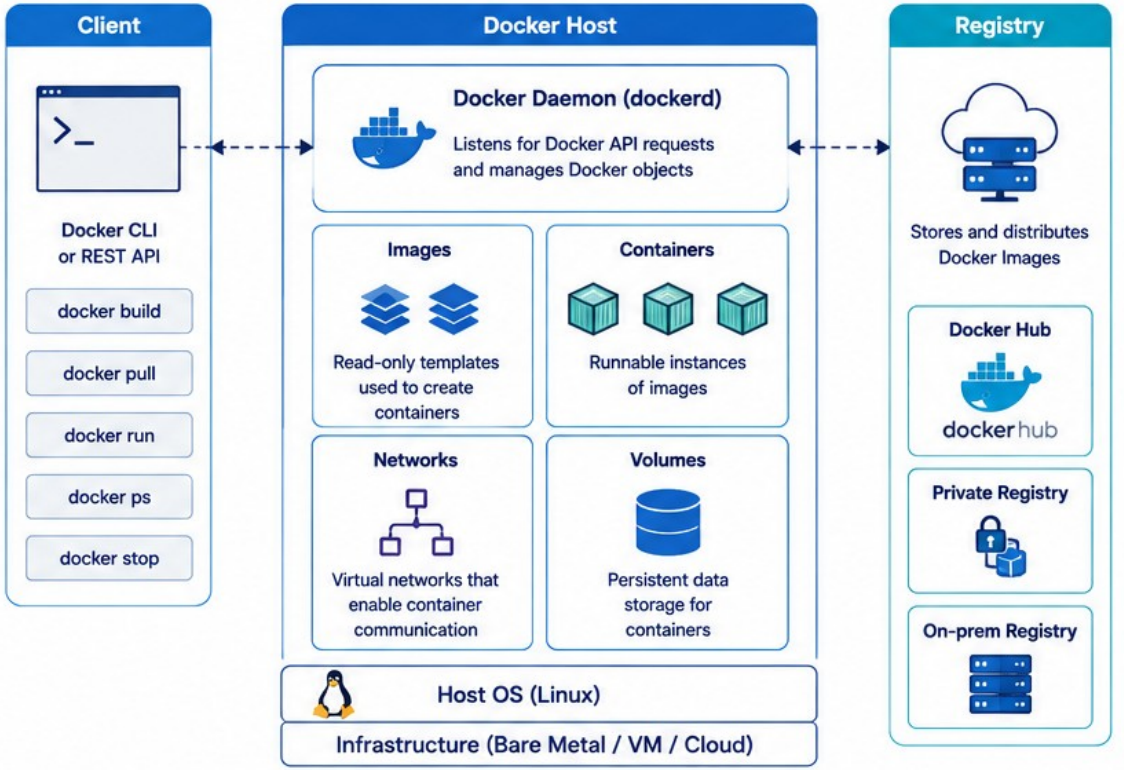
HIGH-LEVEL OVERVIEW

- Docker uses a client–server architecture to build, ship, and run containerized applications.
- The Docker Client communicates with the Docker Daemon.
- The Daemon manages images, containers, networks, and volumes.
- Images are stored in a Registry (Docker Hub or private registry).

KEY BENEFITS

- Lightweight and portable containers
- Consistent environments
- Isolation and resource efficiency
- Scalable and easy to manage
- Rapid deployment and rollback

DOCKER ARCHITECTURE



REGISTRY TYPES

- **Docker Hub** — Public cloud registry provided by Docker
- **Private Registry** — Organization's private image repository
- **On-prem Registry** — Self-hosted registry within your infrastructure

DATA FLOW

1. Client sends commands to Docker Daemon.
2. Dameon builds, pulls, or runs containers using images.
3. Images are pulled from or pushed to the Registry.
4. Containers use Networks and Volumes for communication and persistent storage.

CORE DOCKER COMPONENTS

CLIENT

CLI or API used by users to interact with Docker.

- Send commands
- Build images
- Run containers
- Manage Docker objects

DOCKER DAEMON

Background service that manages Docker objects.

- Handles API requests
- Manages images, containers, networks, volumes
- Enforces policies

IMAGES

Immutable templates used to create containers.

- Built from Dockerfile
- Layered file system
- Versioned and shareable
- Stored in Registry

CONTAINERS

Running instances of images.

- Isolated processes
- Ephemeral by default
- Share OS kernel
- Start, stop, remove

VOLUMES

Persistent data storage managed by Docker.

- Data durability
- Decoupled from container lifecycle
- Backup and restore

NETWORKS

Enable communication between containers and external world.

- Bridge (default)
- Host
- Overlay
- Macvlan

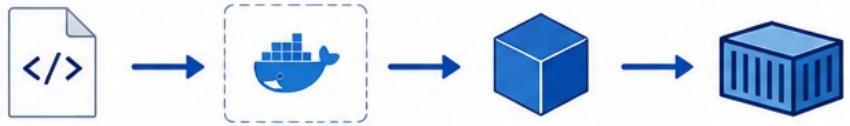
DOCKERFILE REVIEW

A Dockerfile is a text file with instructions to build a Docker image automatically.

WHAT IS A DOCKERFILE?

A Dockerfile contains a set of instructions that Docker uses to build an image layer by layer.

DOCKERFILE WORKFLOW



1. WRITE — Write Dockerfile with instructions.

2. BUILD — Docker builds image layer by layer.

3. IMAGE — Resulting image is created.

4. RUN — Run a container from the image.

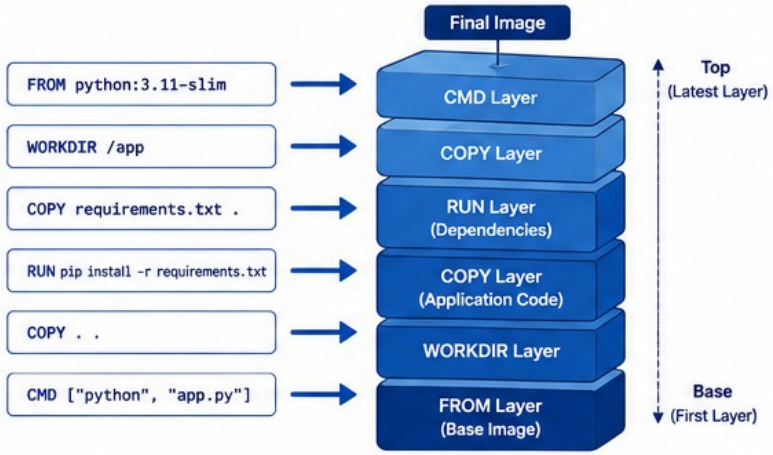
DOCKERFILE EXAMPLE

```dockerfile
# Use an official Python runtime as a parent image
FROM python:3.11-slim

# Set working directory
WORKDIR /app

# Copy requirements and install dependencies
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY . .

# Set the default command
CMD ["python", "app.py"]
```

COMMON INSTRUCTIONS

FROM	Specifies the base image
WORKDIR	Sets the working directory
COPY	Copies files or directories into the image
ADD	Copies files or URLs (with additional features)
RUN	Executes commands during image build
CMD	Sets default command for container execution
ENTRYPOINT	Configures a container that always executes
EXPOSE	Informs Docker that the container listens on ports
ENV	Sets environment variables
ARG	Defines build-time variables
VOLUME	Creates a mount point for persistent data
USER	Sets the user for running the container

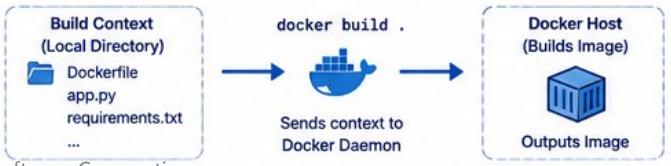
BUILD PROCESS (LAYER BY LAYER)



BEST PRACTICES

- Use official base images
- Keep images small and focused
- Order instructions from least to most frequently changed
- Leverage build cache effectively
- Use `.dockerignore` to exclude unnecessary files
- Run as non-root user
- Scan images for vulnerabilities
- Use specific tags, avoid `latest`

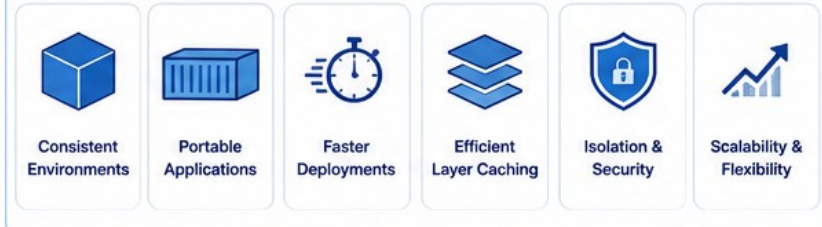
DOCKERFILE BUILD CONTEXT



.DOCKERIGNORE EXAMPLE

```
.git                # Ignore git directory
__pycache__/        # Ignore Python cache
*.pyc               # Ignore Python compiled files
node_modules/       # Ignore Node modules
*.log               # Ignore log files
Dockerfile          # (Optional) exclude itself
```

KEY BENEFITS



DOCKERFILE BEST PRACTICES

Small, deliberate choices in the Dockerfile are what actually make an image safe to run in a shared cluster.

DOCKERFILE RULES

FROM a minimal, maintained base image -- not a bloated general-purpose one.

COPY only what the runtime stage needs -- never the whole build context.

USER set explicitly before the final ENTRYPOINT/CMD.

No secret or credential ever baked into a layer.

COMMON MISTAKES

- Root as the default (implicit) user.
- A single-stage build that ships the JDK and Maven cache.
- ENV lines carrying a plaintext password or API key.
- :latest as the only tag -- no version, no commit SHA.

Build and push (from the lab guide)

```
docker build -t "$REGISTRY/crm-api:${VERSION}-${GIT_SHA}" -f backend/Dockerfile backend
docker push "$REGISTRY/crm-api:${VERSION}-${GIT_SHA}"
```

KEY TAKEAWAY A secret baked into an image layer is exposed forever in that layer's history -- environment injection at runtime is the only safe pattern.

IMAGE OPTIMIZATION BEST PRACTICES

A smaller, better-layered image starts faster, scans faster, and gives an attacker less surface to work with.

OPTIMIZATION TECHNIQUES

Order layers so rarely-changing steps cache first.

Combine related RUN instructions to reduce layer count.

Strip build-only dependencies from the final runtime stage.

Prefer a JRE-only base for the runtime stage over a full JDK.

WHY IT MATTERS HERE

- A smaller image scans faster in the pipeline's image-scan gate.
- Fewer packages means a smaller CVE surface to remediate.
- Faster pulls mean faster, more predictable k3s rollouts.
- Cache-friendly layering keeps CI/CD pipeline time reasonable.

Layer ordering (concept)

```
COPY pom.xml .          # changes rarely -> cached
RUN mvn dependency:go-offline
COPY src ./src          # changes often -> invalidates cache last
```

KEY TAKEAWAY An unoptimized image is not just slow -- every unnecessary package inside it is one more thing the image-scan gate has to evaluate and the team has to justify.

Image Optimization Best Practices

1

Choose the Right Dimensions

Serve images in the exact size needed to avoid unnecessary downloads.

2

Use Modern Image Formats

Prefer AVIF or WebP for better compression and quality. Fall back to JPEG or PNG when needed.

3

Compress Images Efficiently

Reduce file size without noticeable quality loss using the right compression tools and settings.

4

Enable Lazy Loading

Load images only when they enter the viewport to improve initial page performance.

5

Use Responsive Images

Implement srcset and sizes to serve the best image for each device and screen size.

6

Use SVG for Icons and Graphics

Use SVG for scalable vector graphics to keep them sharp and lightweight.

7

Optimize for Performance Metrics

Optimize images to improve Core Web Vitals like LCP (Largest Contentful Paint).

8

Use a CDN

Deliver images via a Content Delivery Network to reduce latency and speed up loading globally.

9

Optimize Metadata

Remove unnecessary metadata (EXIF, comments) to reduce file size and protect privacy.

10

Test and Monitor Regularly

Continuously test image performance and monitor the impact on user experience.

GITHUB ACTIONS PIPELINE REVIEW

The pipeline gate order is fixed: never deploy an artifact that has not been verified, scanned, and published first.

Stage	Purpose
build	Compile the application and produce a build artifact.
verify	Run the full test suite -- mvn clean verify must be green.
scan	Secret, dependency, and image scans -- criticals block unless exceptioned.
publish	Push the immutable, digest-identified image to the registry.
deploy (gated)	Roll out to k3s only after every prior stage passes.

PIPELINE HABITS

Secrets Stay Scoped

Registry and cluster credentials live in Actions secrets, never in YAML.

Identity Flows Forward

The same verified tag/digest moves stage to stage -- no silent rebuild.

Approvals Where Required

A deploy gate can require an explicit human approval, per instructor policy.

Reports Preserved

Test and scan reports are saved as pipeline artifacts, not discarded.

KEY TAKEAWAY A pipeline that deploys before its scan stage completes is not gated -- it is just automated, which is a very different guarantee.

GitHub Actions Pipeline Review

What Triggers the Pipeline

- Push to branches
- Pull Request opened/ synchronized
- Manual dispatch
- Schedule (cron)
- Workflow dispatch

Get the Code

- Uses actions/ checkout
- Fetches the latest code
- Supports submodules

Compile & Build Artifacts

- Install dependencies
- Compile source code
- Run linters & formatters
- Produce build artifacts

Verify Code Quality

- Run unit tests
- Run integration tests
- Generate test reports
- Fail fast on errors

Code Quality & Security

- Static code analysis
- Security scanning
- Dependency scanning
- Quality gate checks

Package Artifacts

- Create Docker image (or other artifacts)
- Tag version
- Push to registry
- Store build artifacts

Deploy to Environments

- Deploy to dev/ staging/prod
- Environment protection rules
- Approvals & gates
- Post-deploy verifications

Best Practices Checklist

- Use specific action versions (pinned)
- Keep secrets in GitHub Secrets
- Use matrix builds for multiple versions
- Cache dependencies for speed
- Fail fast and clear logging
- Use environments and protection rules
- Monitor pipeline health and metrics

BUILD AUTOMATION WORKFLOW

The build stage produces one artifact, deterministically, that every later stage can trust without rebuilding.

BUILD STAGE RESPONSIBILITIES

Resolve dependencies from a locked, reproducible source.

Compile backend (and frontend, if containerized) once.

Fail fast on a compile error -- no partial artifact moves forward.

Tag the resulting artifact with version and commit SHA immediately.

AUTOMATION DISCIPLINE

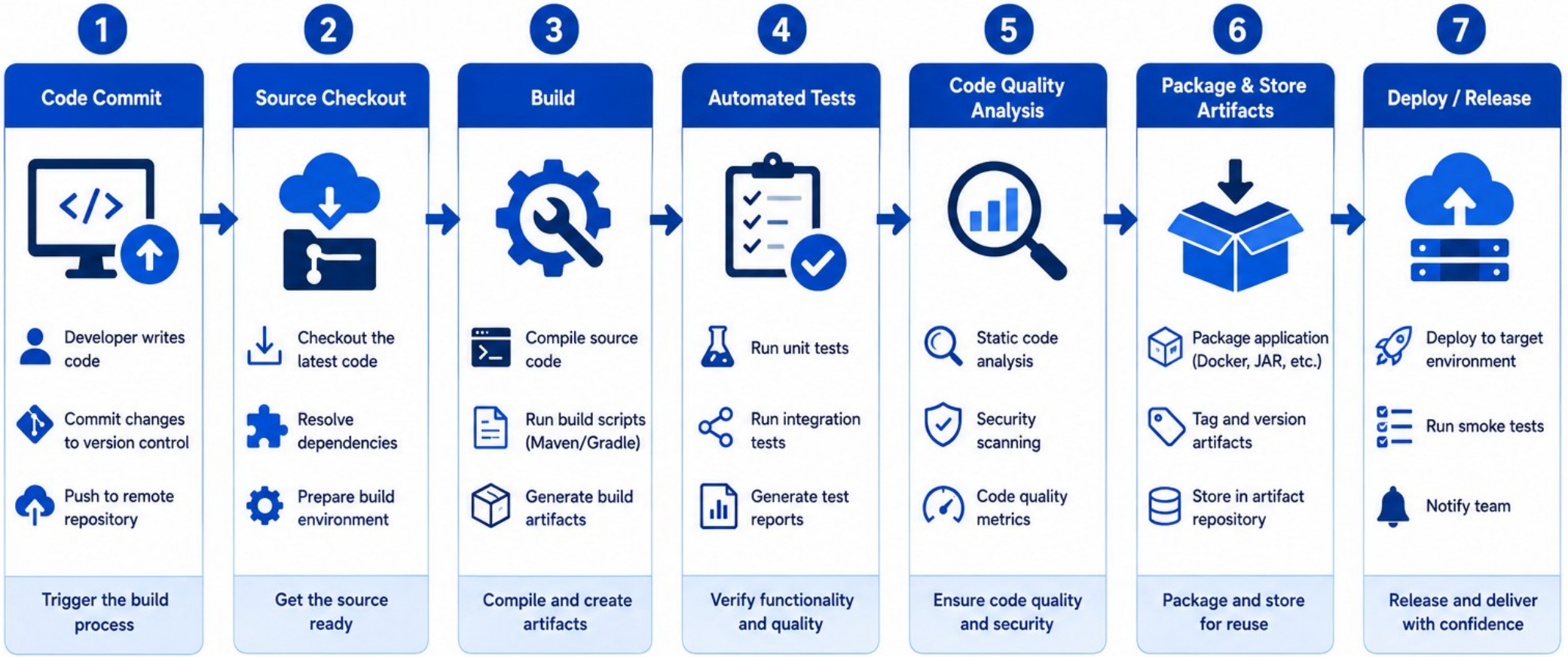
- The same build output is what gets tested, scanned, and published.
- No environment-specific rebuild later in the pipeline.
- Build logs are preserved as an artifact for later review.
- A flaky build stage undermines trust in every stage downstream of it.

Local equivalent (from the lab guide)

```
./mvnw -B clean verify
docker build -t crm-api:local -f backend/Dockerfile backend
```

KEY TAKEAWAY Rebuilding the artifact separately for test, scan, and deploy stages is exactly the provenance-destroying mistake this module's pipeline design prevents.

Build Automation Workflow



Key Benefits

Faster Builds
Automate repetitive steps to save time

Consistent Builds
Standardized process reduces errors

Early Detection
Find issues early in the pipeline

Better Quality
Quality gates ensure reliable releases

Team Productivity
Developers focus more on innovation

TESTING AUTOMATION WORKFLOW

The verify stage runs the exact same test suite Modules 49 and 50 already built -- now enforced as a pipeline gate.

WHAT VERIFY RUNS

Backend: unit, MockMvc, JPA, and Kafka integration tests.

New in this module: AuthorizationTests (401/403/200).

Frontend, if in scope: lint, component tests, and build.

A red test suite stops the pipeline before scan or publish.

AUTOMATION DISCIPLINE

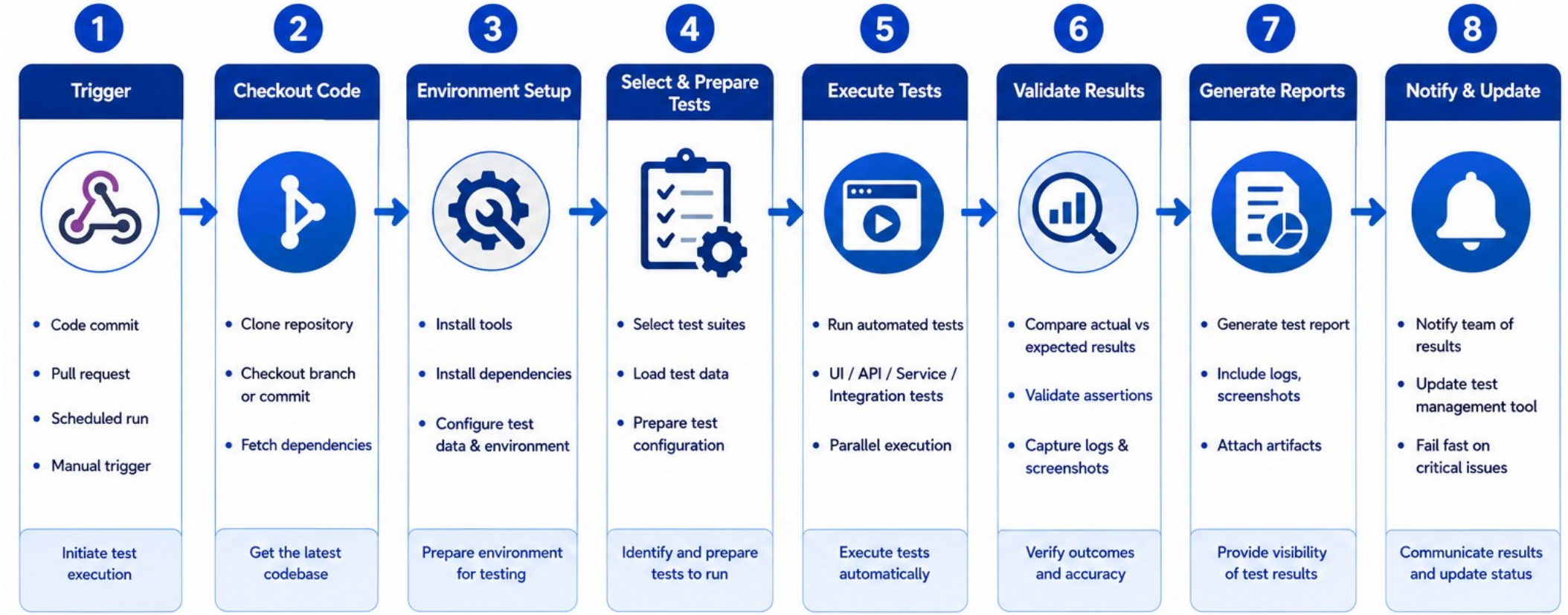
- No test gets skipped or commented out to force a pipeline pass.
- Flaky tests get fixed, not silently retried until green.
- Test reports (Surefire, npm) are preserved as pipeline artifacts.
- The same test suite runs locally and in CI -- no CI-only leniency.

Verify stage (from the lab guide)

```
cd ~/java-bootcamp/examples/customer-management-platform
./mvnw -B clean verify
```

KEY TAKEAWAY A pipeline that skips a failing test to keep moving is functionally identical to having no test gate at all.

Testing Automation Workflow



Key Enablers

Test Frameworks
Selenium, JUnit, TestNG, Cypress, Playwright

Test Data Management
Reliable, maintainable and reusable test data

Environment Management
Consistent and isolated test environments

CI/CD Integration
Automated execution in pipeline

Metrics & Dashboards
Track quality, trends and stability

Quality Gates
Define thresholds and prevent bad releases

SECURITY-SCANNING WORKFLOW

Scan stages run after verify and before publish -- an artifact never reaches the registry unscanned.

SCAN STAGE ORDER

secret-scan runs first -- a leaked credential blocks everything.

dep-scan checks resolved dependencies against known CVEs.

image-build produces the container; image-scan inspects it.

Every scan stage can fail the pipeline, not just log a warning.

SCANNING DISCIPLINE

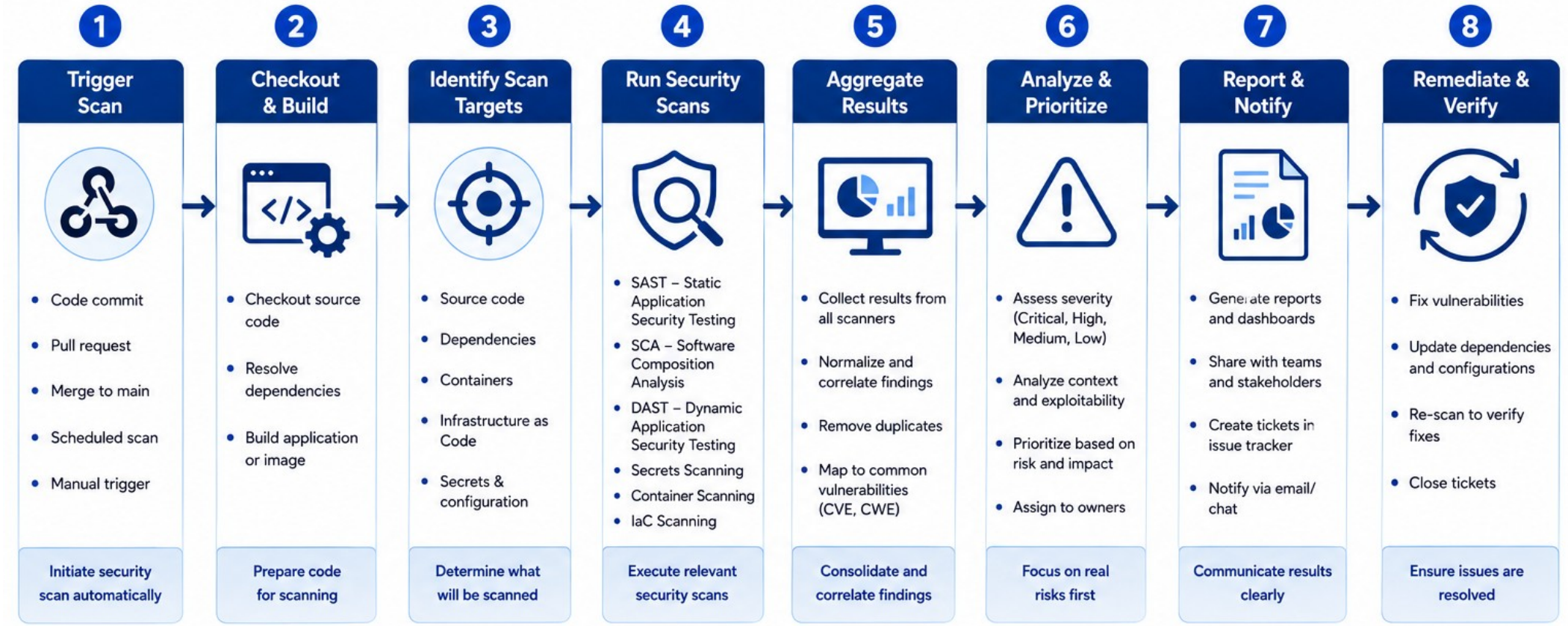
- Sanitized reports get linked from docs/security-deploy-demo.md.
- A critical finding needs a fix or a time-bound, owned exception.
- Scan configuration lives in the pipeline, not a developer's laptop.
- Re-run scans after any dependency or base-image bump.

Stage acceptance note (from the lab guide)

```
| Stage   | Pass condition              | Artifact out       |
| scans   | no unexceptioned criticals  | sanitized reports  |
| publish| push succeeds                | tag + digest       |
```

KEY TAKEAWAY Reordering deploy before scans, even temporarily to 'unblock' a demo, is exactly the gate-skipping shortcut this module's rubric is built to catch.

Security Scanning Workflow



Best Practices

Scan Early & Often
Shift security left and scan at every stage of the SDLC.

Keep Tools Updated
Use the latest versions of scanners and vulnerability databases.

Reduce False Positives
Tune rules and use context to minimize noise.

Risk-Based Approach
Prioritize vulnerabilities based on severity and business impact.

Integrate with DevSecOps
Embed scanning in CI/CD pipeline and development workflow.

Track & Measure
Monitor trends and improve security continuously.

Protect Data
Secure scan data, secrets, and access controls.

TRY IT YOURSELF -- EXERCISE 3: OUTLINE DELIVERY GATES

Reinforces: listing the pipeline stages, a failing SAST requirement, and artifact identity, before writing any workflow YAML.

STEPS (notes/lab51-pipeline-gates.md)

Step	What To Do
1 -- Stages	build, test, SAST/dependency-check, package image, (deploy as authorized).
2 -- Check the Reference	The SAST gate must be able to fail the pipeline, not just log.
3 -- Secrets	Checklist: no credentials in YAML -- use Actions secrets.
4 -- Artifact Identity	Require a digest/checksum recorded for any promotion.

Delivery-gate outline

```
Stages: build -> test -> SAST/dep-scan -> image -> (deploy).
Secrets: Actions secrets only.   Identity: digest recorded before promotion.
```

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 3 before continuing: exercises/exercise-03-pipeline-gates.md (workspace: examples/module-51-exercises).

DEPLOYMENT AUTOMATION WORKFLOW

The deploy stage applies manifests, waits for a healthy rollout, and only then declares success.

DEPLOY STAGE STEPS

Apply k8s/ manifests pinned to the published digest, not a tag.

kubectl rollout status waits for readiness before declaring done.

Environment-scoped secrets are injected, never templated in plain text.

A deploy that never reaches ready state is a pipeline failure, not a warning.

AUTOMATION DISCIPLINE

- Approval gates can require a human before production-like deploy.
- Deploy stage runs only after build, verify, scan, and publish pass.
- Rollout status and events get captured as pipeline log artifacts.
- The same manifest set is used for every deploy -- no hand-edited YAML mid-incident.

Rollout wait (from the lab guide)

```
kubectl apply -f k8s/
kubectl rollout status deployment/crm-api --timeout=180s
```

KEY TAKEAWAY A deploy stage that declares success right after kubectl apply, without waiting for rollout status, ships broken pods with a green checkmark.

Deployment Automation Workflow

1 Trigger Deployment

- Code commit or merge
- Manual trigger
- Scheduled deployment
- Webhook event

Initiate deployment process

2 Validate & Build Artifacts

- Checkout code
- Run build scripts
- Execute tests
- Package application

Ensure build is successful

3 Store Artifacts

- Upload artifacts to repository
- Store container images
- Version and tag artifacts

Store artifacts for reliable retrieval

4 Prepare Environment

- Provision/verify infrastructure
- Configure servers
- Set environment variables
- Run pre-deployment checks

Ensure environment is ready

5 Deploy Application

- Deploy to target environment
- Run database migrations
- Configure application
- Start/restart services

Deploy application automatically

6 Verify Health & Smoke Tests

- Health check endpoints
- Smoke tests
- Validate logs
- Performance checks

Verify applioyment success

7 Rollback on Failure

- Detect deployment failure
- Automated rollback
- Restore previous version
- Notify team

Minimize impact with rollback

8 Notify & Record Deployment

- Notify stakeholders
- Update deployment status
- Record deployment history
- Generate reports

Keep stakeholders informed

KEY ENABLERS

Infrastructure as Code
Automate provisioning and configuration for consistency and repeatability.

Security & Compliance
Use secrets management, access controls, and policy checks.

CI/CD Integration
Integrate with CI/CD pipeline for seamless, automated deployments.

Monitoring & Observability
Monitor application health and performance continuously.

Version Control
Track changes, versions, and releases for traceability and audit.

Collaboration
Improve communication and visibility across teams.

TERRAFORM PROVISIONING REVIEW

Infrastructure as code makes the training cluster's shape reviewable and repeatable -- not a set of manual clicks nobody wrote down.

TERRAFORM REVIEW POINTS

Declarative resources describe the desired cluster/namespace state.

terraform plan is reviewed before terraform apply runs.

State is stored and locked appropriately -- never edited by hand.

Variables carry environment differences -- never hardcoded secrets.

WHY IT MATTERS FOR RELEASE

- A namespace or resource quota drift can silently break a deploy.
- IaC changes go through the same PR review as application code.
- Reproducible infrastructure is what makes Lab 52's evidence trustworthy.
- This module reviews IaC -- it does not require writing new modules from scratch.

Plan-before-apply discipline (concept)

```
terraform plan -out=tfplan   # reviewed by a teammate
terraform apply tfplan      # only the reviewed plan is applied
```

KEY TAKEAWAY An unreviewed terraform apply run directly against the shared training cluster is exactly the kind of undocumented change this review discipline exists to prevent.

TERRAFORM PROVISIONING REVIEW

1. WHAT IS TERRAFORM?

Terraform is an Infrastructure as Code (IaC) tool that lets you define, provision, and manage cloud and on-prem resources in a declarative way.

2.. KEY BENEFITS

- Consistent & Repeatable Infrastructure
- Version Controlled Infrastructure
- Faster Provisioning & Scaling
- Cost Efficient Resource Management
- **Multi-Cloud & Provider Agnostic**

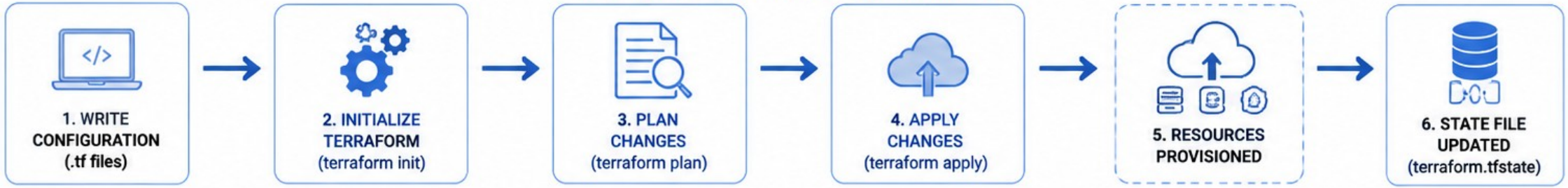
3.. CORE CONCEPTS

- Configuration Files (HCL)
- Resources (What to create)
- Providers (Cloud/API plugins)
- Variables (Input values)
- State File (Tracks real resources)

4. PROVISIONING FLOW

1. Write Terraform Configuration (.tf files)
2. Initialize Terraform (terraform init)
3. Review Changes (terraform plan)
4. Create/Update Resources (terrafarm apply)
5. Manage & Maintain (terrafarm destroy / plan / apply)

TERRAFORM PROVISIONING WORKFLOW



5. COMMON PROVIDERS

AWS · Azure · Google Cloud · HashiCorp · Kubernetes

6. BEST PRACTICES

- Use Remote State
- Version Control Code
- Use Modules
- Secure Sensitive Data
- Validate with terraform plan
- Use Workspaces for Environments
- Tag Resources
- Automate with CI/CD

7. KEY COMMANDS

terraform init	Initialize working directory
terraform plan	Preview changes
terraform apply	Apply changes
terraform destroy	Destroy resources
terraform fmt	Format configuration files
terraform validate	Validate configuration

ANSIBLE CONFIGURATION REVIEW

Configuration management keeps cluster nodes and shared tooling consistent -- reviewed here, not authored from scratch.

ANSIBLE REVIEW POINTS

Playbooks are idempotent -- running twice produces the same end state.

Inventory separates environments; no shared secrets across them.

Vault (or an equivalent secret manager) protects sensitive variables.

Configuration drift gets corrected by re-running the playbook, not a manual SSH fix.

WHY IT MATTERS FOR RELEASE

- A manually patched node is undocumented and unreproducible.
- Consistent baseline configuration reduces surprising deploy failures.
- Playbook changes are reviewed, matching the Terraform review discipline.
- This module reviews the pattern -- it does not require writing new playbooks.

Idempotency check (concept)

```
ansible-playbook site.yml   # first run: makes changes
ansible-playbook site.yml   # second run: reports zero changes
```

KEY TAKEAWAY A node fixed manually over SSH during an incident, with no playbook update to match, drifts from the documented baseline the moment anyone forgets exactly what they did.

ANSIBLE CONFIGURATION REVIEW

1. WHAT IS ANSIBLE?

ANSIBLE

Ansible is an agentless IT automation tool that helps you configure, manage, and deploy applications across servers using simple, human-readable YAML.

2. KEY BENEFITS

- **Agentless Architecture**
  No agents to install or manage
- **Simple & Easy to Learn**
  YAML based playbooks
- **Consistent & Repeatable**
  Automate configurations reliably
- **Idempotent Operations**
  Ensures systems stay in desired state
- **Scalable & Flexible**
  Manage from few to thousands of nodes

3. CORE COMPONENTS

- **Inventory**
  Defines managed hosts and groups
- **Playbooks**
  YAML files with tasks, variables, handlers
- **Modules**
  Pre-built units to perform tasks
- **Roles**
  Organize playbooks, tasks and files
- **Variables**
  Store values used in playbooks
- **Facts**
  Information gathered from managed hosts

4. ANSIBLE CONFIGURATION FLOW

1. Define Inventory (hosts & groups)
2. Write / Update Playbook
3. Run Playbook (ansible-playbook)
4. Execute Tasks on Managed Hosts
5. Validate & Verify Configuration
6. Repeat / Automate

ANSIBLE CONFIGURATION WORKFLOW

1. INVENTORY
Define hosts and groups

→

2. PLAYBOOK
Create or update playbook (YAML)

→

3. VARIABLES
Define variables and values

→

4. EXECUTION
Run playbook (ansible-playbook)

→

5. MANAGED HOSTS
Tasks executed on target hosts

→

6. VERIFICATION
Validate configuration and state

→

7. REPEAT
Maintain desired state continuously

5. COMMON MODULE CATEGORIES

- **System** — Manage OS and system settings
- **Services** — Manage and control services
- **Files** — Manage files and directories
- **Networking** — Manage network configuration
- **Users** — Manage users and groups
- **Cloud** — Manage cloud resources
- **Packages** — Install and manage packages
- **Database** — Manage database tasks

6. BEST PRACTICES

- Use Version Control (Git)
- Keep Playbooks Small & Modular
- Use Roles for Reusability
- Use Variables & Vault for Secrets
- Test in Non-Production First
- Follow Idempotent Practices
- Use Tags for Selective Runs
- Document Playbooks
- Automate with CI/CD
- Monitor and Audit Changes

7. KEY ANSIBLE COMMANDS

COMMAND	DESCRIPTION
ansible all -m ping	Test connectivity to all hosts
ansible all -m setup	Gather facts from all hosts
ansible-playbook site.yml	Run a playbook
ansible-playbook site.yml --check	Perform a dry run (no changes)
ansible-playbook site.yml --diff	Show changes without applying
ansible-vault encrypt file.yml	Encrypt a file with Ansible Vault
ansible-galaxy install role_name	Install a role from Ansible Galaxy

INFRASTRUCTURE VALIDATION

Before trusting a deploy target, prove the cluster context, namespace, and access are actually what the team expects.

VALIDATION CHECKS

kubectl config current-context confirms the intended cluster.

Namespace confirmed before any manifest is applied.

Registry credentials and pull access verified ahead of deploy.

kubectl version --client confirms tooling compatibility.

VALIDATION DISCIPLINE

- Never commit a kubeconfig file to Git, ever.
- Record the namespace in docs -- never assume everyone remembers it.
- A wrong-context deploy is a named, avoidable incident, not bad luck.
- Validate before every deploy, not just the first one of the day.

Pre-flight validation (from the lab guide)

```
kubectl config current-context
kubectl config view --minify -o jsonpath='{.contexts[0].context.namespace}{"\n"}'
```

KEY TAKEAWAY Registry or authentication issues at this validation step often burn 45-60 minutes unverified -- checking pull secrets early is time well spent.

INFRASTRUCTURE VALIDATION

1. PURPOSE

Infrastructure validation ensures that the deployed infrastructure meets the required standards, configurations, security policies, and performance expectations before it is used in production.

2. KEY BENEFITS

- **Ensures Compliance**
  Meets security and regulatory standards
- **Improves Reliability**
  Verifies infrastructure stability and resilience
- **Reduces Risk**
  Identifies misconfigurations early
- **Optimizes Performance**
  Validates capacity, scalability and efficiency
- **Ensures Consistency**
  Confirms infrastructure matches design and baseline

3. VALIDATION CATEGORIES

- **Configuration Validation**
  Check settings against baseline and standards
- **Security Validation**
  Verify security controls, access, encryption, and hardening
- **Performance Validation**
  Validate capacity, latency, throughput, and resource usage
- **Connectivity Validation**
  Ensure network, DNS, routing, and service connectivity
- **Availability & Resilience**
  Test HA, failover, backups, and disaster recovery
- **Cost Validation**
  Validate cost estimates and resource utilization

4. VALIDATION METHODS / TOOLS

- **Automated Tests**
  Infrastructure as Code checks, policy as code
- **Command Line Checks**
  CLI, scripts, and custom validations
- **Monitoring & Observability**
  Metrics, logs, alerts, and dashboards
- **Security Scans**
  Vulnerability scans, compliance scans, CIS benchmarks
- **Configuration Management**
  Drift detection and compliance verification
- **Chaos / Resilience Testing**
  Failure injection and recovery tests

5. INFRASTRUCTURE VALIDATION WORKFLOW

1. **DEFINE**
   Define requirements, standards, and success criteria
2. **PREPARE**
   Review architecture, configurations, and baseline
3. **EXECUTE**
   Run validation tests using tools and automation
4. **ANALYZE**
   Analyze results, identify issues and gaps
5. **REMEDIATE**
   Fix issues and update configurations as needed
6. **RE-VALIDATE**
   Re-run tests to verify issues are resolved
7. **DOCUMENT**
   Document results, approvals, and evidence
8. **APPROVE**
   Obtain sign-off for production readiness

6. VALIDATION CHECKLIST (EXAMPLES)

✓ All resources created as per design	✓ DNS resolution working
✓ Configurations match baseline	✓ Load balancers and health checks OK
✓ Security groups / firewall rules validated	✓ Monitoring and alerts configured
✓ IAM roles and permissions validated	✓ Backup and restore tested
✓ Encryption enabled (in transit & at rest)	✓ Auto-scaling and capacity validated
✓ Patching and updates up to date	✓ Logs are collected and accessible
✓ Network connectivity verified	✓ Compliance policies validated

7. COMMON VALIDATION TOOLS

Terraform	Ansible	AWS Config	Azure Policy
Puppet	Chef	Inspec	Nessus

8. VALIDATION OUTCOMES

- Infrastructure is secure, reliable, and compliant
- Performance and capacity are validated
- Risks and misconfigurations are resolved
- Production-ready infrastructure with confidence
- Documentation and approvals are recorded

TRY IT YOURSELF -- EXERCISE 4: FILL DEPLOY EVIDENCE TODOS

Reinforces: naming exactly what deploy evidence will be captured, before the real rollout happens.

STEPS (notes/lab51-deploy-evidence-todos.md)

Step	What To Do
1 -- Template	Fill blanks: digest, namespace, probes, smoke result, rollback digest.
2 -- Manifest Names	List the k8s/ files expected: deployment, service, route/ingress.
3 -- Probe Paths	Note readiness/liveness paths against the real actuator config.
4 -- Scope	TODO sheet only -- the real deploy happens during Lab 51.

Deploy evidence skeleton

```
Digest: _____   Namespace: _____   Probes: readiness/liveness paths noted.
Smoke: 401 anonymous + 200 authenticated.   Rollback digest: _____
```

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 4 before continuing: exercises/exercise-04-deploy-evidence-todos.md (workspace: examples/module-51-exercises).

KUBERNETES DEPLOYMENT REVIEW

A Deployment, a Service, and health probes -- the minimum shape a workload needs to run safely in a shared cluster.

CORE OBJECTS

Deployment manages replica pods and rollout history.

Service gives the workload a stable network identity inside the cluster.

Route/Ingress exposes the service beyond the cluster, if required.

ConfigMap/Secret inject configuration and credentials, never baked into the image.

REVIEW DISCIPLINE

- Readiness and liveness probes are required, not optional.
- Resource requests/limits are set deliberately -- never left default.
- Image reference is pinned to a digest, not a mutable tag.
- kubectl rollout history keeps prior revisions available for rollback.

Probe sketch (from the lab guide)

```
readinessProbe: { httpGet: { path: /actuator/health/readiness, port: 8080 } }
livenessProbe:  { httpGet: { path: /actuator/health/liveness,  port: 8080 } }
```

KEY TAKEAWAY A Deployment with no readiness probe sends live traffic to a pod that has not actually finished starting -- probes are what prevent that.

KUBERNETES DEPLOYMENT REVIEW

1. PURPOSE

Kubernetes deployment manages the rollout and lifecycle of applications in a consistent, scalable, and resilient manner across a cluster.

2. KEY BENEFITS

- **Scalability** — Scale applications up or down easily
- **High Availability** — Self-healing and rolling updates minimize downtime
- **Consistency** — Declarative state ensures consistency across environments
- **Resource Efficiency** — Optimized resource utilization and scheduling
- **Portability** — Run workloads anywhere

3. KEY COMPONENTS

- **Deployment** — Manages desired state for Pods and updates
- **Pod** — Smallest deployable unit containing one or more containers
- **Service** — Stable network endpoint for accessing Pods
- **Node** — Worker machine in the cluster where Pods run
- **ConfigMap / Secret** — Manage configuration and sensitive data
- **Ingress** — Manage external access to services

4. DEPLOYMENT STRATEGIES

- **Rolling Update** — Gradually update Pods with zero downtime
- **Recreate** — Terminate all existing Pods before creating new ones
- **Blue-Green Deployment** — Switch traffic between blue (old) and green (new) environments
- **Canary Deployment** — Roll out to a small subset of users before full release

5. KUBERNETES DEPLOYMENT WORKFLOW

1. **DEFINE** — Define application requirements, resources, and configuration
2. **MANIFEST** — Create Kubernetes manifests (Deployment, Service, etc.)
3. **APPLY** — Apply manifests using kubectl or CI/CD pipeline
4. **SCHEDULE** — Kubernetes schedules Pods on appropriate nodes
5. **RUN** — Containers start and application is running
5. **MONITOR** — Monitor health, logs, and performance
7. **UPDATE / SCALE** — Update or scale application as needed
8. **MAINTAIN** — Ensure reliability, backups, and continuous operations

6. BEST PRACTICES

- [x] Use Rolling Updates for zero downtime
- [x] Set resource requests and limits
- [x] Use health checks (Liveness & Readiness Probes)
- [x] Store configuration in ConfigMaps and Secrets
- [x] Implement proper logging and monitoring
- [x] Use Namespaces to isolate environments
- [x] Enable RBAC for secure access
- [x] Test deployments in staging before production

7. DEPLOYMENT REVIEW CHECKLIST

- [x] Manifests are valid and up-to-date
- [x] Resource requests and limits set
- [x] Health checks configured
- [x] Images are from trusted registry
- [x] Secrets are encrypted and safe
- [x] Services have correct selectors
- [x] Ingress rules are correct
- [x] Network policies are applied
- [x] Pods scheduled on correct nodes
- [x] Rolling update strategy configured
- [x] Replica count matches requirements
- [x] Monitoring and alerts configured
- [x] Logs are centralized and accessible
- [x] Backup and disaster recovery in place
- [x] RBAC permissions configured
- [x] CI/CD pipeline integrated and tested

8. COMMON KUBERNETES COMMANDS

COMMAND	DESCRIPTION
kubectl apply -f <file.yaml>	Apply configuration from a file
kubectl get pods	List all Pods
kubectl get deployments	List all Deployments
kubectl describe <resource> <name>	Show details of a resource
kubectl logs <pod-name>	View logs of a Pod
kubectl scale deployment <name> --replicas=<n>	Scale a Deployment
kubectl rollout status deployment/<name>	Check rollout status
kubectl rollout undo deployment/<name>	Rollback to previous version
kubectl delete -f <file.yaml>	Delete resources from a file

K3S DEPLOYMENT REVIEW WITH OPENSHIFT COMPARISON

The capstone deploys to k3s -- name OpenShift as the enterprise-standard comparison point, the same dual-technology pattern Module 42 used.

Aspect	k3s (capstone target)	OpenShift (enterprise comparison)
Footprint	Lightweight, single-binary Kubernetes distribution.	Full enterprise Kubernetes platform with added tooling.
Routing	Ingress (or a simple LoadBalancer/NodePort).	Route objects, a first-class OpenShift concept.
Image policy	Digest-pinned manifests, enforced by team discipline.	Built-in ImageStreams and trigger-based rollout policy.
Security defaults	Configured explicitly per manifest.	Security Context Constraints enforced platform-wide.
Where it runs here	The actual training cluster this capstone deploys to.	Named for comparison only -- not deployed to in this lab.

WHY THIS COMPARISON MATTERS

1. **Same Pattern as Module 42**
   Dual-technology coverage: hands-on with one, literacy with its enterprise sibling.
2. **k3s Is the Real Target**
   Every deploy, smoke test, and rollback in Lab 51 runs against k3s, not OpenShift.
3. **OpenShift Is Interview-Relevant**
   Many enterprises standardize on OpenShift -- recognizing its concepts matters.
4. **Concepts Transfer**
   Deployments, Services, and probes mean the same thing on both platforms.

KEY TAKEAWAY State clearly: the capstone itself deploys to k3s. OpenShift is named here as the enterprise-standard comparison point, not a second deployment target for this lab.

k3s DEPLOYMENT REVIEW WITH OPENSHIFT COMPARISON

1. k3s OVERVIEW

k3s

Lightweight, certified Kubernetes distribution designed for edge, IoT, and small environments. Packaged as a single binary with minimal dependencies.

- Lightweight & Fast
- Low Resource Usage
- Easy Installation
- Edge & IoT Ready
- Kubernetes Compatible

2. OPENSHIFT OVERVIEW

Red Hat OpenShift

Enterprise Kubernetes platform by Red Hat with developer tools, security, CI/CD, and management built-in for cloud, hybrid, and large-scale deployments.

- Enterprise Grade
- Built-in Security
- Developer Productivity
- Hybrid & Multi-Cloud
- Full Platform Services

3. OPPLOYMENT WORKFLOW

k3s DEPLOYMENT

1. Install k3s (Single Binary)
2. Configure (Cluster & Nodes)
3. Deploy Workloads (kubectl apply)
4. Expose Services (NodePort / Ingress)
5. Monitor & Maintain (k3s Tools)

OPENSHIFT DEPLOYMENT

1. Install OpenShift (Installer / OCP)
2. Configure Cluster (Infra & Operators)
3. Deploy Workloads (oc apply / Helm)
4. Expose Services (Route / Ingress)
5. Monitor & Maintain (OpenShift Console)

4. KEY COMPARISON

FEATURE	k3s	OPENSHIFT
Architecture	Kubernetes Distro	Enterprise Platform
Installation	Single Binary	Installer / OCP
Cluster Size	Small to Medium	Medium to Large
Resource Usage	Very Lightweight	Higher (Feature Rich)
Ease of Setup	Very Easy	Moderate
Management	CLI (kubectl)	Web Console + CLI (oc)
Built-in Services	Minimal	Extensive (Logging, Monitoring, Registry, etc.)
Security	K8s Native	Enterprise Security (RBAC, SCC, PSA)
Updates	Simple (k3s upgrade)	Controlled (OCP Update)
Best Use Cases	Edge, IoT, Dev/Test, Small Clusters	Enterprise, Production, Hybrid / Multi-Cloud

5. DEPLOYMENT REVIEW CHECKLIST

k3s

k3s CHECKLIST

- k3s installed and running on all nodes
- Nodes joined and ready (kubectl get nodes)
- Network (CNI) is healthy
- CoreDNS running and resolving
- Storage (local / Longhorn) configured
- RBAC and access configured
- Workloads deployed and running
- Services accessible
- Ingress / NodePort working
- Monitoring and logging configured
- Backups and upgrades tested

OPENSHIFT CHECKLIST

- OpenShift cluster installed and healthy
- All cluster operators are available
- Nodes schedulable and ready
- Storage (CSI) provisioned
- Projects / Namespaces created
- RBAC / SCC / PSA configured
- Applications deployed (oc get pods)
- Routes / Ingress accessible
- Monitoring, Logging, Alerting configured
- Registry accessible and images stored
- Backups and disaster recovery tested

6. WHEN TO USE WHAT?

USE k3s WHEN

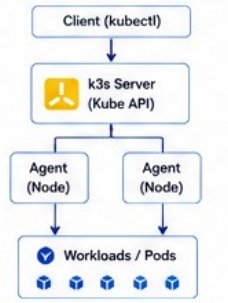
- You need a lightweight cluster
- Edge / IoT / Remote locations
- Limited CPU, memory, storage
- Quick setup and easy management
- Cost-sensitive environments
- Dev / Test / Learning environments

USE OPENSHIFT WHEN

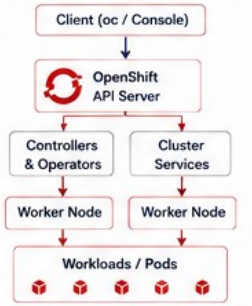
- Enterprise production workloads
- Need advanced platform services
- Require strong security & compliance
- Hybrid / Multi-cloud strategy
- Large teams & complex applications
- Need web and upgrades & redistors

7. ARCHITECTURE COMPARISON

k3s ARCHITECTURE



OPENSHIFT ARCHITECTURE



8. KEY DIFFERENCES AT A GLANCE

k3s

- Lightweight Kubernetes distribution
- Single binary installation
- Minimal components
- Ideal for edge & small clusters
- Lower resource consumption
- Basic management via CLI

OPENSHIFT

- Full enterprise platform
- Operator-based architecture
- Extensive built-in services
- Designed for scale & HA
- Higher resource consumption
- Advanced management (Console + CLI)

9. SUMMARY

k3s is lightweight, simple, and ideal for edge, IoT, and small clusters.

OpenShift is enterprise-grade with advanced features, security, and management.

Choose **k3s** for simplicity and speed.

Choose **OpenShift** for enterprise scale and capabilities.

SCALING STRATEGY

More replicas only helps if the application can actually run more than one instance safely at once.

SCALING CONSIDERATIONS

The API must be stateless -- no in-memory session tied to one pod.

Kafka consumer group semantics already handle multiple consumer instances.

Database connection pool sizing must account for multiple replicas.

Horizontal scaling is preferred over a single, larger pod.

SCALING DISCIPLINE

- Never scale to hide a performance problem instead of fixing it.
- Resource requests must be accurate for scaling decisions to make sense.
- Readiness probes matter more, not less, as replica count grows.
- Document the tested replica count -- do not just assume it scales.

Scale command (concept)

```
kubectl scale deployment/crm-api --replicas=3
kubectl get pods -l app=crm-api -w
```

KEY TAKEAWAY A stateful assumption baked into the API silently breaks the moment a second replica starts -- statelessness is what makes scaling safe at all.

SCALING STRATEGY

1. WHAT IS SCALING?

Scaling is the ability of a system to handle growing workloads by increasing capacity, maintaining performance, availability, and reliability cost-effectively.

2. KEY OBJECTIVES

- Handle Increased Load
- Maintain Performance
- Ensure High Availability
- Optimize Cost
- Ensure Reliability

3. SCALING TYPES

VERTICAL SCALING
(Scale Up / Down)

- Add more CPU, RAM, Storage to existing node
- Simple to implement
- Limited by hardware
- Downtime may be required

HORIZONTAL SCALING
(Scale Out / In)

- Add more instances/nodes
- High availability & fault tolerant
- More complex
- Preferred for large scale

4. SCALING DIMENSIONS

- **Compute** — CPU / Memory
- **Storage** — Capacity / IOPS
- **Network** — Bandwidth / Connections
- **Application** — Instances / Pods / Containers
- **Users / Workload** — Requests / Traffic

5. SCALING TRIGGERS

- High CPU / Memory Usage
- High Response Time
- High Request Rate
- Queue Backlog
- Storage Utilization
- Business Growth

6. SCALING STRATEGIES & PATTERNS

AUTO SCALING
- Automatically adjust capacity based on metrics
- Scale Up on demand Scale Down when not needed

LOAD BALANCING
- Distribute traffic across instances
- Improve performance and availability

REPLICATION
- Replicate instances or data
- Improve availability and read throughput

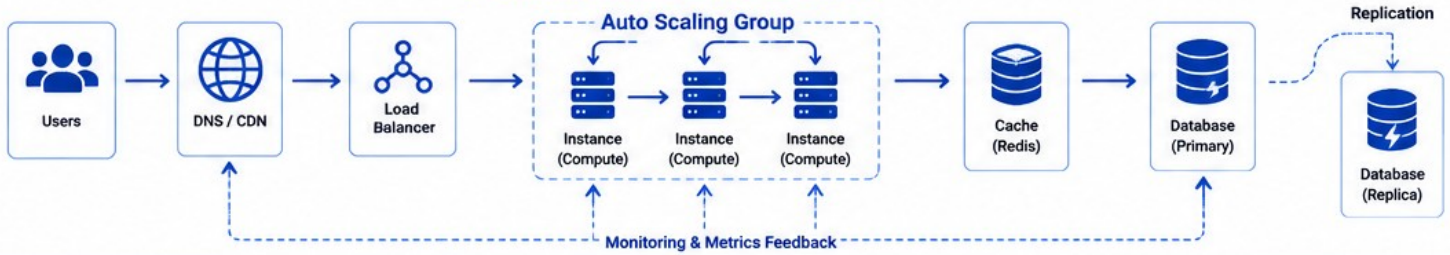
SHARDING / PARTITIONING
- Split data across multiple nodes
- Improve write throughput and storage capacity

CACHING
- Cache frequently accessed data
- Reduce load on backend systems

MICROSERVICES
- Scale individual services independently
- Use right-sized resources per service

CONTENT DELIIVER NETWORK (CDN)
- Distribute content globally
- Reduce latency and origin server load

7. SCALING ARCHITECTURE EXAMPLE



8. BEST PRACTICES

- Design for scalability from the start
- Monitor metrics and set alerts
- Use automation (Auto Scaling)
- Load test regularly
- Optimize code and queries
- Use caching and CDN
- Choose right instance types
- Plan for failure and redundancy
- Review and optimize costs

9. SCALING APPROACH COMPARISON

APPROACH	HOW IT WORKS	PROS	CONS	BEST SUITED FOR
Vertical Scaling (Scale Up)	Increase resources of a single node	Simple, quick, lower complexity	Limited by hardware, downtime possible	Small to medium workloads, quick needs
Horizontal Scaling (Scale Out)	Add more nodes/instances	High availability, fault tolerant, flexible	More complex, requires load balancing	Large scale, dynamic workloads
Auto Scaling	Automatically scale based on metrics	Cost-effective, handles spikes	Needs proper metrics & configuration	Variable workloads, cloud environments
Microservices Scaling	Scale individual services independently	Efficient resource use, flexible	Complex architecture & operations	Large applications with many services

HIGH AVAILABILITY STRATEGY

Availability is what happens when one pod, one node, or one dependency fails -- plan for it deliberately.

AVAILABILITY BUILDING BLOCKS

Multiple replicas mean one pod failing does not take the API down.

Readiness probes remove an unhealthy pod from rotation automatically.

A rolling update keeps the API available during a deploy, not just after.

Kafka consumer rebalancing tolerates a consumer instance restarting.

AVAILABILITY DISCIPLINE

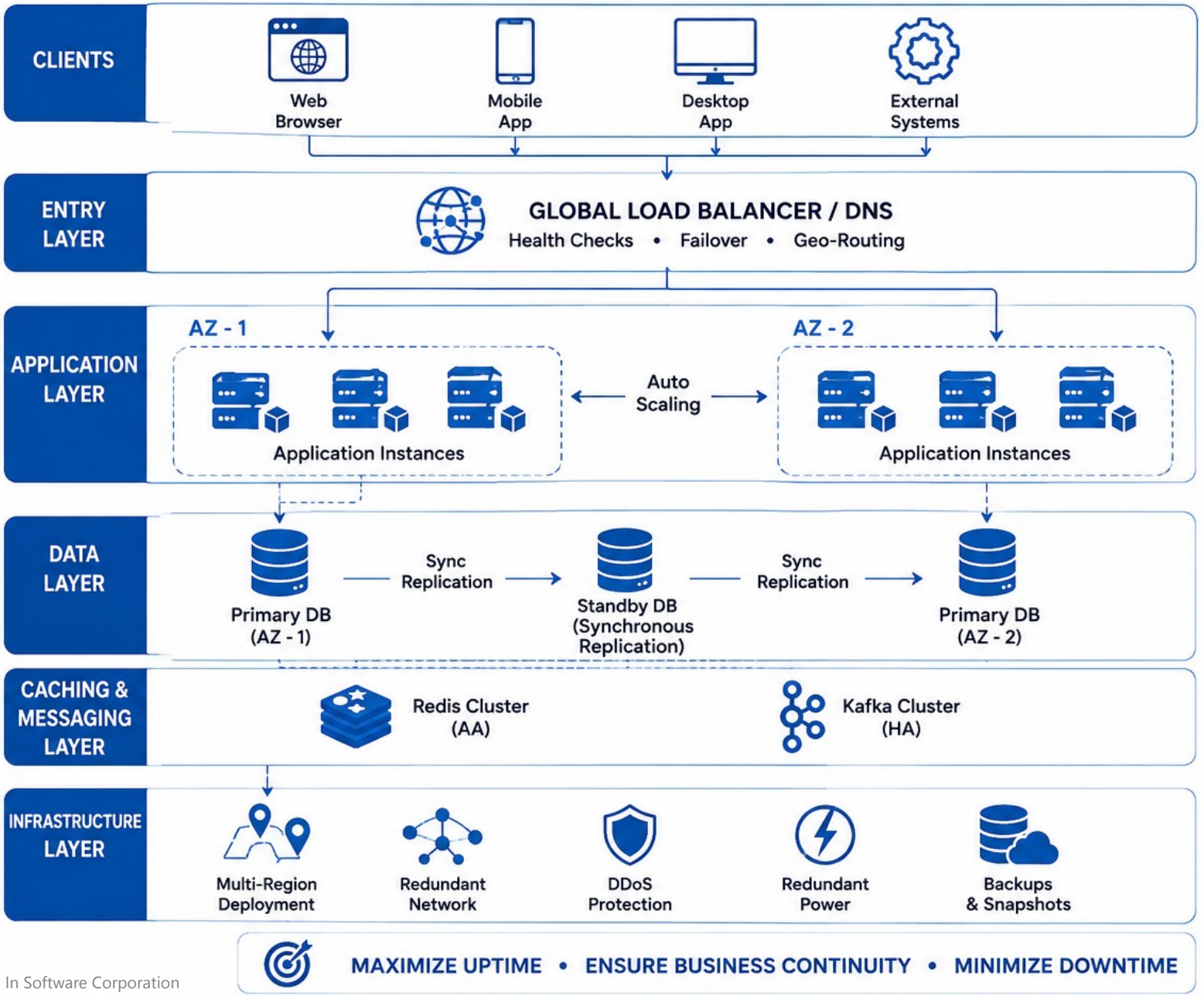
- PostgreSQL availability is a separate, instructor-managed concern in training.
- A single point of failure gets named explicitly, not assumed away.
- Availability claims need the same evidence bar as any other claim.
- A rollback plan is itself part of the availability story, not an afterthought.

Rolling update behavior (concept)

```
# Deployment default strategy: RollingUpdate
# old pods terminate only as new, ready pods take their place
```

KEY TAKEAWAY An availability slide with no named single point of failure has not actually analyzed availability -- it has only asserted it.

HIGH AVAILABILITY STRATEGY



HA PRACTICES

- Health Monitoring & Alerting
- Automatic Failover
- Auto Scaling
- Zero / Minimal Downtime Deployments
- Regular Backup & DR Testing
- Security & Compliance

DISASTER RECOVERY

Cross-Region Backup → Failover to DR Region

DEPLOYMENT CHECKLIST

Before applying a manifest to the shared cluster, confirm every one of these is actually true.

Digest Pinned

The manifest references the exact scanned, published digest -- never a tag alone.

Probes Configured

Readiness and liveness probes point at the real actuator paths.

Secrets Wired

Registry pull secret and app secrets are mounted, never templated in plain text.

Context Confirmed

kubectl context and namespace verified before kubectl apply runs.

KEY TAKEAWAY Skipping this checklist to save time is exactly how a CrashLoopBackOff or a wrong-namespace deploy becomes a live-demo surprise during Module 52.

DEPLOYMENT CHECKLIST

1 PRE-DEPLOYMENT

- [ ] Confirm requirements are met
- [ ] Review and approve change request
- [ ] Ensure all tests are passed
- [ ] Code reviewed and merged
- [ ] Verify environment readiness
- [ ] Check dependencies and integrations
- [ ] Database changes reviewed
- [ ] Rollback plan prepared
- [ ] Stakeholders notified
- [ ] Backup (if required) completed

2 BUILD & PACKAGE

- [ ] Code compiled successfully
- [ ] Unit tests passed
- [ ] Artifacts built successfully
- [ ] Version / build number updated
- [ ] Configuration files validated
- [ ] Secrets and credentials verified
- [ ] Package scanned for vulnerabilities
- [ ] Artifacts stored in repository

3 PRE-DEPLOY CHECKS

- [ ] Target environment verified
- [ ] Disk space and resources verified
- [ ] Service accounts and permissions verified
- [ ] Configurations validated
- [ ] Feature flags reviewed
- [ ] Health check endpoints verified
- [ ] Monitoring and logging configured
- [ ] Smoke test plan reviewed
- [ ] Rollback steps confirmed

4 DEPLOYMENT

- [ ] Deploy to target environment
- [ ] Monitor deployment progress
- [ ] Run database migrations (if any)
- [ ] Verify services started successfully
- [ ] Check application health
- [ ] Run smoke tests
- [ ] Validate integrations
- [ ] Verify logs for errors
- [ ] Confirm metrics are normal

5 POST-DEPLOYMENT

- [ ] Validate functionality
- [ ] Run end-to-end tests
- [ ] Verify performance and metrics
- [ ] Check alerts and notifications
- [ ] Update documentation
- [ ] Notify stakeholders
- [ ] Monitor for issues
- [ ] Confirm backup completed
- [ ] Close change request

6 ROLLBACK READINESS

- [ ] Rollback plan easily accessible
- [ ] Previous version available
- [ ] Database rollback script ready
- [ ] Configuration rollback steps documented
- [ ] Team aware of rollback process
- [ ] Test rollback steps periodically

PLAN • VERIFY • DEPLOY • VALIDATE • MONITOR

SMOKE TESTING

Both halves of the smoke test matter equally -- the authenticated success path and the anonymous denial path.

SMOKE TEST SEQUENCE

Health check: actuator/health/readiness returns healthy.

Anonymous /api/customers returns exactly 401.

Authenticated read with a valid token returns 200.

Correlation id release-smoke-${BUILD} traceable in logs.

SMOKE DISCIPLINE

- Skipping the unauthorized check is a named failure-handling deduction.
- Smoke runs against the pinned digest, never a floating tag.
- A flaky smoke result gets investigated, not silently retried until green.
- Smoke output gets saved as sanitized evidence, tokens redacted.

Smoke sequence (from the lab guide)

```
curl -fsS "$CRM_URL/actuator/health/readiness"
test "$(curl -s -o /dev/null -w '%{http_code}' "$CRM_URL/api/customers")" = "401"
curl -fsS "$CRM_URL/api/customers?page=0&size=1" -H "Authorization: Bearer $SMOKE_TOKEN"
```

KEY TAKEAWAY Smoke that only proves the happy path passed is exactly half the evidence this module actually requires.

SMOKE TESTING

Quick verification to check if the build is stable enough for further testing.

OVERVIEW

Smoke Testing is a preliminary level of testing performed on a new build to ensure that the most critical functionalities are working and the build is stable for further testing.

OBJECTIVES

- Verify build stability
- Detect show-stopper defects
- Ensure critical functions work
- Decide whether to proceed to further testing

CHARACTERISTICS

- Quick & Shallow
- Covers critical functionality only
- Mostly automated
- Executed on every build
- Go / No-Go decision

SMOKE TESTING VS SANITY TESTING

ASPECT	SMOKE TESTING	SANITY TESTING
Purpose	Check build stability & critical functions	Verify recent changes or bug fixes
Scope	Broad & shallow	Narrow & deep
When	After every build	After minor changes or bug fix
Who	QA Team	Developers / QA
Automation	Highly automated	Can be manual or automated
Decision	Go / No-Go for further testing	Confirm fix / changes

SMOKE TESTING PROCESS

1. **Build Received** — New build is available for testing.
2. **Test Selection** — Select critical test cases covering major features.
3. **Test Execution** — Execute selected test cases (mostly automated).
4. **Result Evaluation** — Evaluate results and check for failures.
5. **Go / No-Go Decision** — If pass → proceed to further testing; if fail → stop & fix.

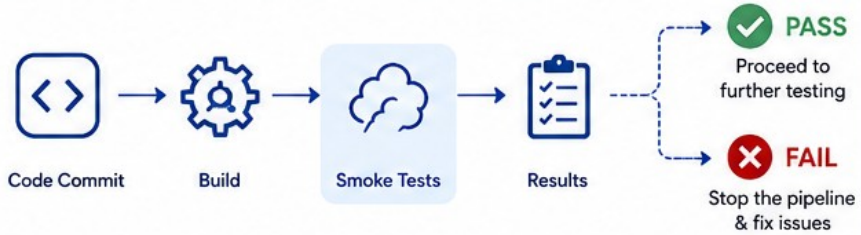
EXAMPLE SMOKE TEST CASES

- Application launch / Login
- Home page / Dashboard load
- Navigation across major modules
- Create / Read critical data
- Search functionality
- Logout
- APIs: Health check / Ping

BEST PRACTICES

- Automate smoke test cases
- Keep test cases small, stable and independent
- Focus on critical functionalities
- Run smoke tests in the test environment
- Integrate with CI/CD pipeline
- Fail fast and provide clear feedback

SMOKE TESTING IN CI/CD PIPELINE



BENEFITS

- Saves time and resources
- Identifies critical issues early
- Improves build quality
- Increases confidence to proceed

TRY IT YOURSELF -- EXERCISE 5: ROLLBACK AND SMOKE MINI-RUNBOOK

Reinforces: writing the exact commands for a rollback and a smoke re-check, before ever needing them under pressure.

STEPS (notes/lab51-rollback-smoke.md)

Step	What To Do
1 -- Trigger	Name the trigger and the decision owner for a rollback.
2 -- Check the Reference	Rollback targets a previous digest -- never 'just restart the pod'.
3 -- Commands	Write the exact kubectl commands for rollback and post-rollback smoke.
4 -- Scope	Mini-runbook only -- the real rehearsal happens during Lab 51.

Rollback mini-runbook skeleton

```
Trigger: failed smoke or CrashLoop.   Owner: on-call lead.
Commands: kubectl rollout undo deployment/crm-api; re-run smoke; confirm digest.
```

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 5 before continuing: exercises/exercise-05-rollback-smoke.md (workspace: examples/module-51-exercises).

MONITORING AND HEALTH CHECKS

Health probes prove a pod is ready right now -- monitoring proves the system stays healthy over time.

WHAT TO MONITOR

Readiness/liveness probe status over time, not just at deploy moment.

Application logs, searchable by correlation id.

Kafka consumer lag, if a consumer is deployed as part of this slice.

Basic resource usage against configured requests/limits.

MONITORING DISCIPLINE

- A metric with no alert is just a number nobody is watching.
- Correlation ids must be searchable without exposing bearer tokens.
- Kafka lag ignored with no consumer-metrics check is a named troubleshooting gap.
- Monitoring evidence belongs in the same demo doc as smoke evidence.

Log search pattern (concept)

```
kubectl logs deployment/crm-api | grep 'release-smoke-'
# confirm the correlation id appears -- confirm no bearer token does
```

KEY TAKEAWAY Kafka lag left unmonitored is a specific, named troubleshooting gap in this module's materials -- add a lag check before declaring the deploy complete.

MONITORING AND HEALTH CHECKS

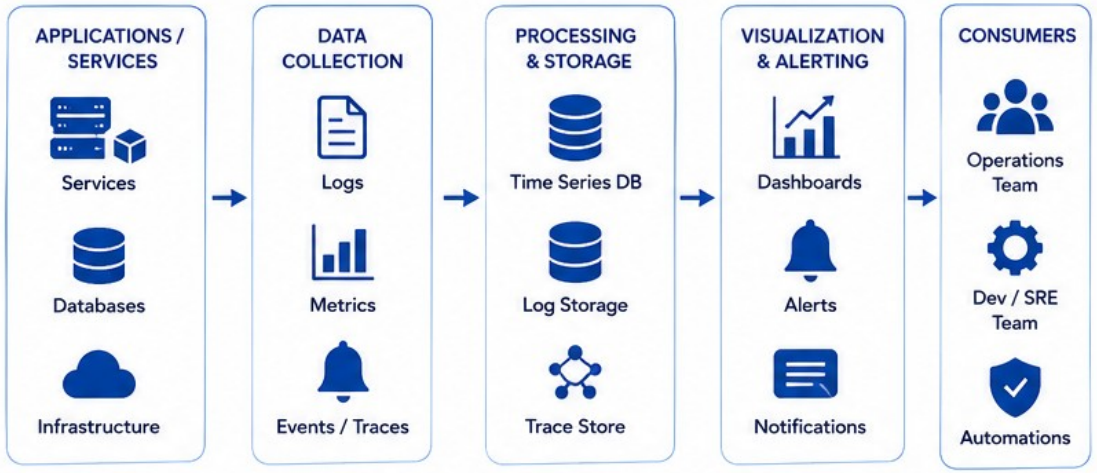
WHAT & WHY

Monitoring
Continuously collect, analyze and visualize metrics, logs and events to understand system behavior.

Health Checks
Actively verify the availability and correctness of services and dependencies.

Goal
Detect issues early, ensure reliability, and minimize downtime.

MONITORING ARCHITECTURE



KEY METRICS

- **Availability (Uptime %)** — Overall system availability
- **Response Time (Latency)** — Time taken to respond
- **Throughput (RPS / TPS)** — Requests or transactions per second
- **Error Rate (%)** — Percentage of failed requests
- **Resource Utilization** — CPU, Memory, Disk, Network
- **Queue / Backlog Size** — Length of message queues
- **Business Metrics** — Revenue, Signups, Orders, etc.

HEALTH CHECKS

Liveness Check
Is the service running?
If not, restart the service.

`GET /health/live`

Readiness Check
Is the service ready to handle traffic?
If not, stop sending traffic.

`GET /health/ready`

Dependency Check
Are dependencies (DB, Cache, API, MQ) accessible?

`GET /health/deps`

Business Check
Is the system working as expected?

`GET /health/business`

ALERTING STRATEGY

1. Define SLOs / Thresholds
2. Detect Anomaly / Breach
3. Trigger Alerts
4. Notify On-call / Channels
5. Acknowledge & Diagnose
6. Resolve & Close

MONITORING DATA

Metrics
Numeric time-series data (e.g., CPU, Latency)

Logs
Discrete events and system logs

Traces
Request flow across microservices

Events
State changes and business events

BEST PRACTICES

- Monitor what matters
- Set sensible thresholds
- Automate health checks
- Alert on symptoms, not noise
- Dashboards for everyone
- Review & improve continuously

COMMON TOOLS

Prometheus · Grafana · Loki · Alertmanager
ELK Stack · Datadog · New Relic · Jaeger

RELEASE APPROVAL PROCESS

A release is approved on evidence, not on confidence -- the same four artifacts get checked every time.

Digest Identity

Tag, digest, pipeline run id, and git SHA all recorded together.

Smoke Evidence

Both 401 anonymous and 200 authenticated proven against the deployed digest.

Scan Disposition

Every critical finding fixed or exceptioned, with an owner and expiry.

Rollback Rehearsed

A previous digest redeploy verified, with health confirmed after.

KEY TAKEAWAY A release approved on 'it looked fine when I checked' instead of these four artifacts is exactly the gap Module 52's evidence-index.md is designed to expose.

RELEASE APPROVAL PROCESS

1 RELEASE PREPARATION

- Complete development and testing
- Update documentation
- Prepare release notes
- Verify all quality gates

2 RELEASE SUBMISSION

- Submit release package
- Include test results, metrics, and checklist
- Identify risks and rollback plan

3 TECHNICAL REVIEW

- Review code changes
- Validate test results
- Assess technical risks
- Verify dependencies and compatibility

4 QUALITY ASSURANCE

- Validate test coverage
- Verify compliance and standards
- Confirm security scans
- Approve quality status

5 STAKEHOLDER REVIEW

- Review business impact
- Validate user readiness
- Confirm communication plan
- Obtain stakeholder approval

6 CHANGE ADVISORY BOARD (CAB)

- Evaluate overall risk
- Review deployment plan
- Ensure rollback readiness
- Approve or reject release

7 FINAL APPROVAL & SIGN-OFF

- Record approvals
- Update release status
- Communicate decision
- Proceed to deployment

ROLES & RESPONSIBILITIES

RELEASE MANAGER
Coordinates the process and owns the release end-to-end.

DEVELOPMENT TEAM
Builds the release and provides artifacts, test results, and notes.

QA TEAM
Validates quality, test coverage, and ensures release readiness.

STAKEHOLDERS
Reviews business impact and confirms readiness and acceptance.

CAB
Reviews risk and approves or rejects the release.

OPERATIONS TEAM
Prepares deployment and ensures operational readiness.

APPROVAL DECISION OUTCOMES

APPROVED	All criteria met. Release is approved and moves to deployment.
CONDITIONAL APPROVAL	Minor issues identified. Action items must be addressed before deployment.
REJECTED	Critical issues or high risks. Release is put on hold until resolved.

KEY ARTIFACTS

- Release Package (Artifacts)
- Test Results & Quality Report
- Release Notes
- Risk Assessment & Rollback Plan
- Release Checklist & Approval Record

RIGHT RELEASE, RIGHT TIME, RIGHT QUALITY. | REDUCE RISK | ENSURE QUALITY | DELIVER VALUE

TRY IT YOURSELF -- EXERCISE 6: RELEASE READINESS SCORECARD

Reinforces: self-scoring release readiness against the four approval artifacts before Lab 51 begins.

STEPS (notes/lab51-prep-checklist.md)

Step	What To Do
1 -- Score Card	Self-rate readiness: threat model, JWT, scans, digest, smoke, rollback.
2 -- Check the Reference	A release is approved on evidence, not on confidence.
3 -- Gaps	Name any item still missing before starting the graded lab.
4 -- Scope	Self-check only -- Lab 51 closes the actual gaps.

Readiness scorecard skeleton

```
Threat model: Ready/Not.   JWT deny-by-default: Ready/Not.
Scans: Ready/Not.   Digest: Ready/Not.   Smoke: Ready/Not.   Rollback: Ready/Not.
```

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 6 before continuing: exercises/exercise-06-release-readiness.md (workspace: examples/module-51-exercises). Final gate before Lab 51.

LAB OVERVIEW -- NORTHSTAR CRM RELEASE GATE

Make the CRM releasable: JWT hardening, gated security scans, an immutable image, k3s deployment, smoke tests, and a rehearsed rollback.

BUSINESS SCENARIO

- The integrated CRM cannot ship until security and delivery gates pass. Reviewers freeze: no 'deployed' claim without digest identity, smoke (auth + deny), health evidence, and a rehearsed rollback.
- You own the release gate for the Week 6 platform, using Amina and Ravi fixtures in smoke tests only.
- A peer must be able to reproduce every result from docs/security-deploy-demo.md alone, without verbal coaching.

LEARNING OBJECTIVES

- Apply JWT authorization with deny-by-default request matching.
- Protect secrets, headers, and actuator exposure.
- Build gated CI/CD stages with verified artifacts.
- Publish immutable multi-stage non-root images.
- Deploy safely to Kubernetes (k3s) with probes.
- Execute authenticated and unauthorized smoke tests, and prove rollback.

WHAT YOU BUILD

- customer-management-platform/: SecurityConfig, AuthorizationTests, a multi-stage Dockerfile, .github/workflows/ci.yml, k8s/ manifests, docs/security-deploy-demo.md, and docs/threat-model.md.

KEY TAKEAWAY Fixtures CUS-1001 (Amina), CUS-1002 (Ravi), and lab-request-001 appear in smoke only -- never real customer data. Duration: about 45 minutes with starters, up to 6-8 hours full path. Advanced capstone difficulty.

LAB TASKS AND DELIVERABLES

lab51-crm -- implementation steps, deliverables, and the evaluation rubric.

#	Implementation Step
1	Threat-model the release: assets, actors, abuse cases, and controls.
2	Secure HTTP endpoints: JWT resource server, deny-by-default.
3	Harden the application: CORS, actuators, logging, security headers.
4	Run security gates: tests, dependency/secret/image scans.
5	Build and publish once: multi-stage, non-root, digest recorded.
6	Create the delivery pipeline: build, verify, scan, publish, gated deploy.
7	Deploy and verify: probes, logs, metrics, authenticated and denied smoke.
8	Prove recovery: rehearsed rollback to a previous digest.

EXPECTED DELIVERABLES

- Spring Security changes and authorization tests
- Pipeline definition
- Dockerfile and image digest record
- Deployment manifests (k3s)
- Security and deployment evidence (scans, smoke, rollback)

RUBRIC (100 MARKS)

- Environment/Structure 10 * Core Impl 30 * Integration/Config 15 * Failure Handling 15 * Verification 10 * Security/Production Awareness 10 * Docs 10

KEY TAKEAWAY Floating :latest loses integration marks. Skipping unauthorized smoke is a failure-handling deduction. Committed secrets require remediation before scoring.

MODULE 51 SUMMARY

In this module, we made the Northstar CRM releasable: hardened, scanned, digest-identified, deployed, smoke-tested, and rollback-proven.

KEY CONCEPTS COVERED

JWT Deny-by-Default

Anonymous -> 401, wrong role -> 403, correct role -> 200 -- all tested.

Layered Hardening

CORS, actuator exposure, and log redaction -- beyond just authentication.

Gated Security Scans

SAST, dependency, secret, and image scans -- criticals fixed or exceptioned.

Immutable Image

Multi-stage, non-root, digest-identified -- never a floating :latest.

Gated CI/CD to k3s

Build -> verify -> scan -> publish -> gated deploy, with health probes.

Rollback Rehearsed

Both smoke halves proven; a previous digest redeploy verified for real.

MODULE FLOW RECAP

1. Threat Model
2. Harden + Enforce
3. Scan + Build
4. Deploy + Smoke
5. Rehearse Rollback

KEY TAKEAWAY A release is graded end to end: authorization, hardening, scanning, image provenance, deployment, smoke, and rollback -- all seven, not just a working deploy.

KNOWLEDGE CHECK (1 OF 2)

Test your understanding of JWT enforcement, hardening, and scanning.

1. What should an anonymous request to a protected /api/ endpoint return?**

A. 200 with limited data

B. 401

C. 403

D. 302 redirect to login

2. What must happen when a security scan finds a critical vulnerability?

A. It is silently ignored to keep the pipeline green

B. It is fixed, or given a time-bounded, owned exception

C. It is logged only, with no further action

D. The scanner is disabled for that stage

3. Why is a floating :latest tag rejected as the only image identity?

A. It is slower to pull than a versioned tag

B. It cannot prove which exact code is actually running

C. Docker does not support it technically

D. It is only a style preference

4. What must a Dockerfile set explicitly to avoid running as root?

A. ENTRYPOINT

B. USER

C. WORKDIR

D. LABEL

KEY TAKEAWAY Anonymous requests get 401; critical scan findings get fixed or exceptioned; digests, not tags, prove identity; USER sets a non-root runtime user.

KNOWLEDGE CHECK (2 OF 2)

Four more questions on the smoke sequence, k3s, rollback, and release approval.

5. What does a complete smoke test sequence require, per this module's rule?

A. Only the authenticated 200 path

B. Only the anonymous 401 path

C. Both the anonymous 401 path and the authenticated 200 path

D. No smoke test is required if unit tests pass

6. What does the capstone actually deploy to, per this module's own statement?

A. OpenShift, exclusively

B. k3s -- OpenShift is named only as an enterprise comparison point

C. A public cloud provider not covered in the course

D. No real cluster -- deployment is simulated

7. What is the correct target for a rollback, per this module's rule?

A. Restarting the current pod

B. A previous, known-good image digest

C. Deleting the namespace and starting over

D. Switching to a different cluster entirely

8. What is required before a release can be approved, per this module's rule?

A. A working deploy alone

B. Digest identity, smoke (auth + deny), scan disposition, and a rehearsed rollback

C. Only a passing unit test suite

D. Verbal confirmation that 'it looks fine'

KEY TAKEAWAY Smoke requires both auth and deny paths; the capstone deploys to k3s; rollback targets a digest; release approval requires all four evidence artifacts together.

"No 'deployed' claim without digest identity, smoke evidence, and a rehearsed rollback."

MODULE 51 COMPLETE

Capstone Security, CI/CD and Deployment

You can now harden JWT authorization to deny-by-default, run security-scanning gates, publish an immutable digest-identified image, deploy it to k3s with health probes, prove authenticated and denied smoke paths, and rehearse a rollback -- the full release gate for the Northstar CRM.